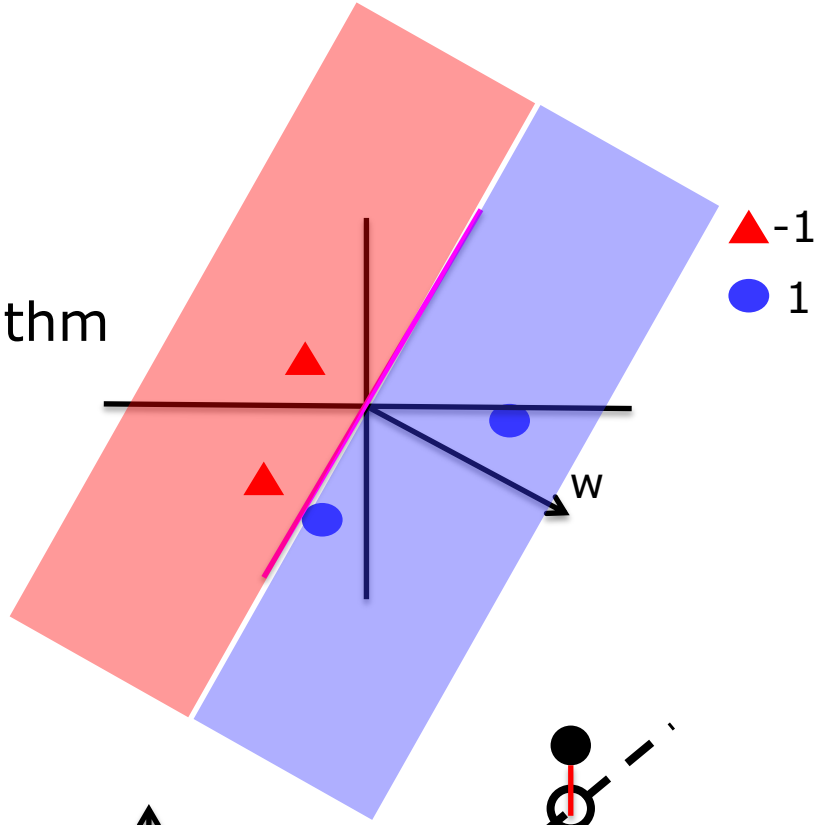
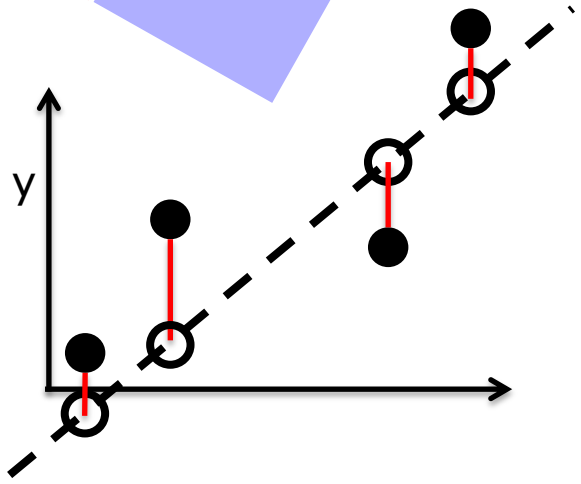
Logistic Regression

Linear Models

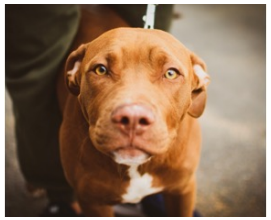
- Classification:
  - Predict(x)=sign($w^T x$)
  - (Pocket) Perceptron Learning Algorithm
  - Y={-1,1}

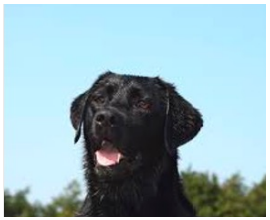


- Regression:
  - Predict(x)=$w^T x$
  - Linear Regression:
    - $w=(X^T X)^{-1} X^T y$
  - Y=R



Linear Models



- Classification:
  - Y={-1,1}
  - Binary decisions.
    - Approve/Decline mortgage?
    - Picture of cat or dog?



- Regression:
  - Y=R
  - Predict values.
    - Price of house.
    - Value of a stock tomorrow.
    - How large a mortgage to allow customer.

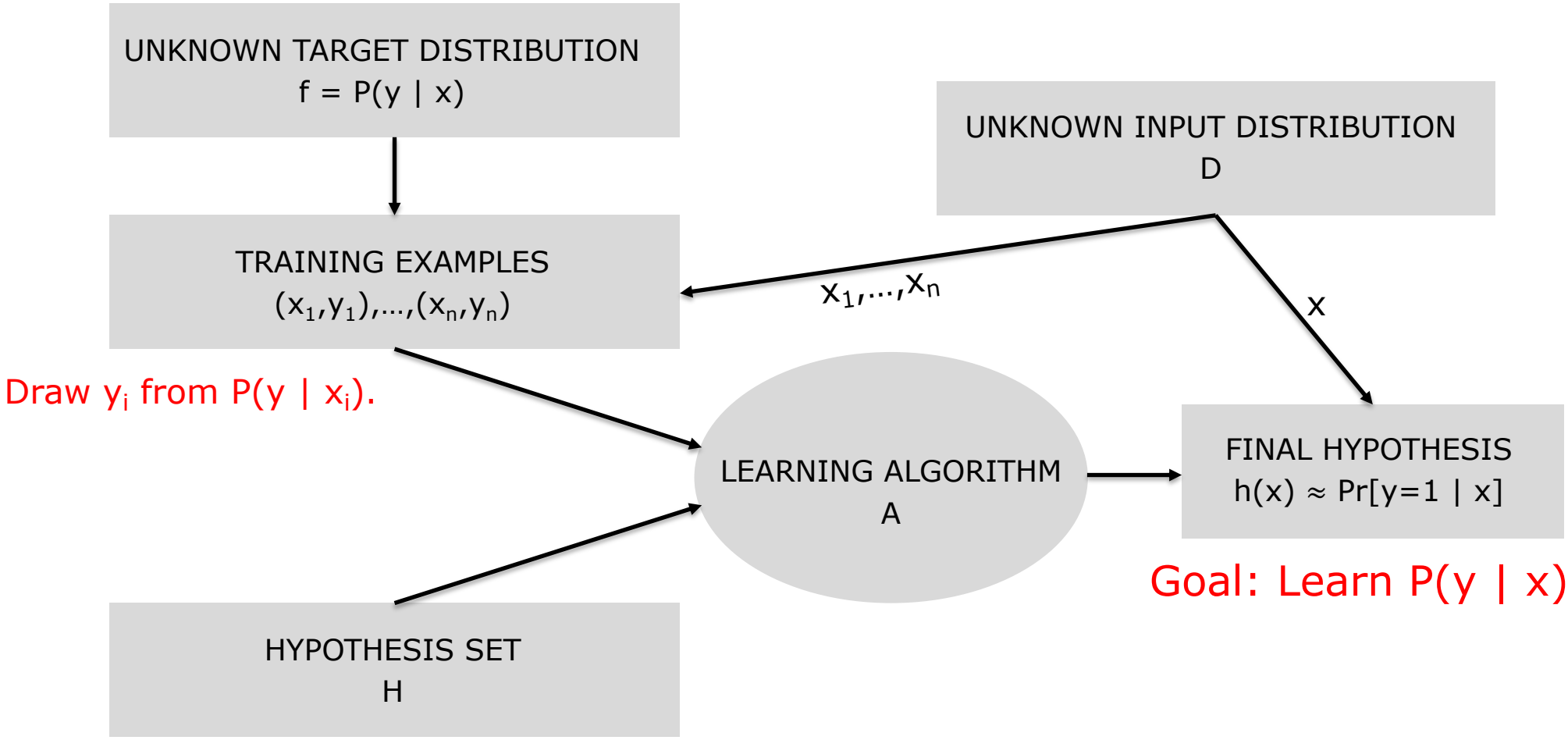
Linear Models

- Logistic Regression:
  - Some applications not properly captured.

  - Probability of heart attack given medical conditions.
  - Probability of failing to pay rates on mortgage.
  - Probability of committing a crime if not imprisoned while awaiting trial.

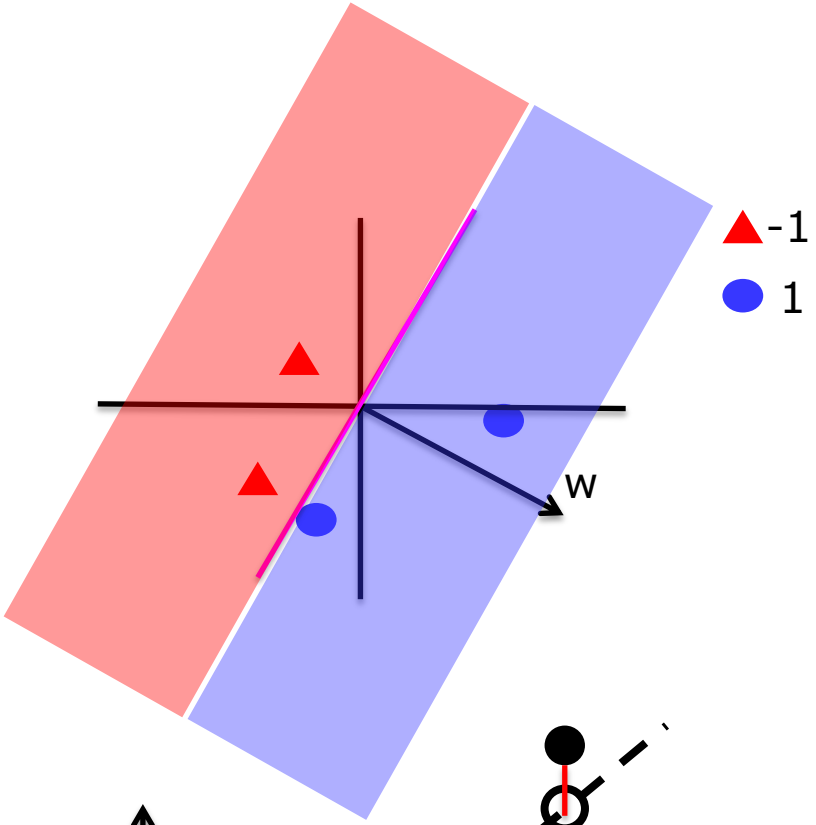
- Setup:
  - Two classes {-1,1}
  - Noisy target (not deterministic) P(y | x).
  - Goal: Learn Pr[y=1 | x], i.e. learn P(y | x).
  - Y=[0,1].

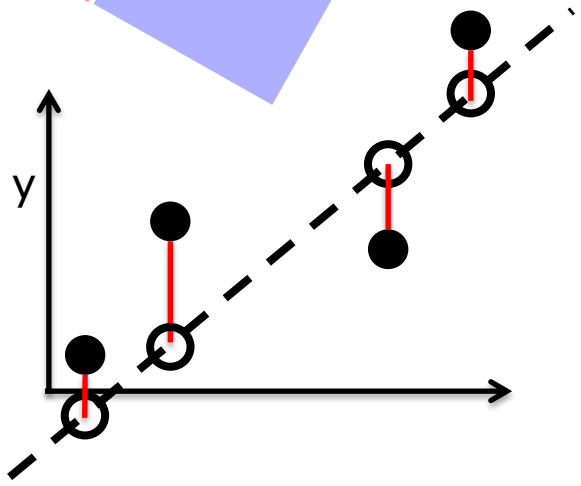
Logistic Regression



Linear Models

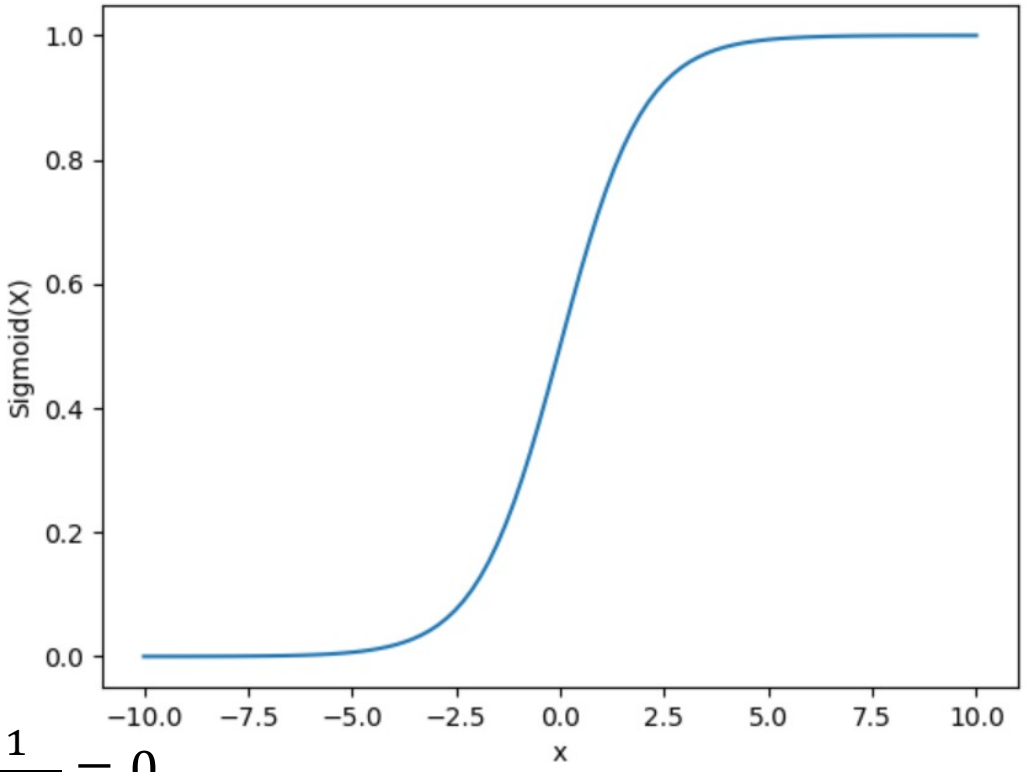
- Classification:
  - Predict(x)=sign($w^T x$)
- Regression:
  - Predict(x)=$w^T x$
- Logistic Regression:
  - Predict(x)= $\theta(w^T x)$??
  - Y=[0,1]
  - How do we make a linear model with outputs in [0,1]?





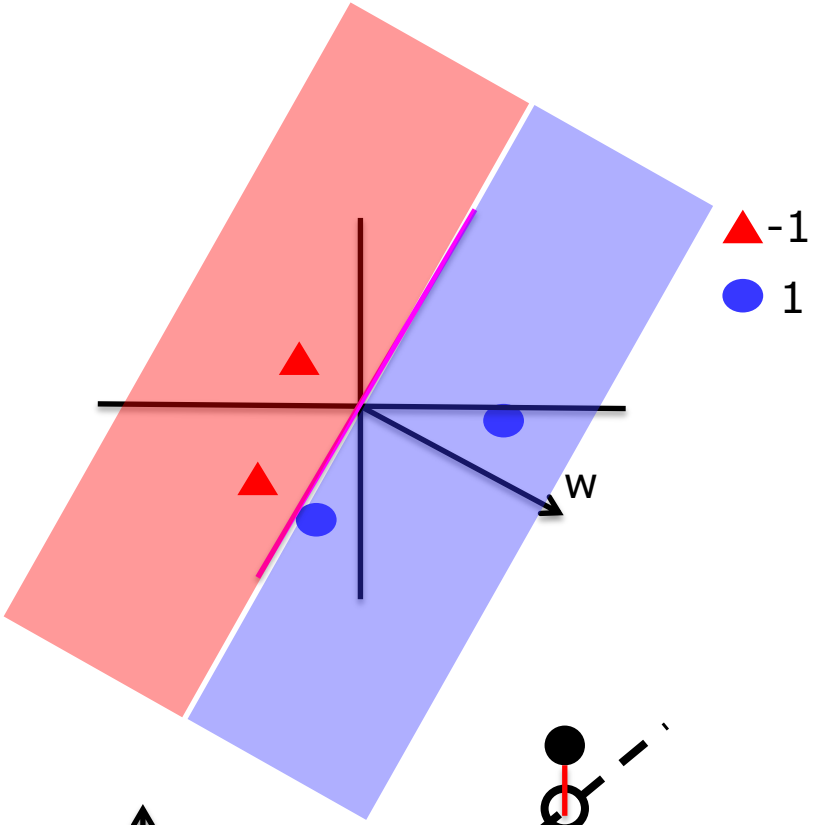
Sigmoid

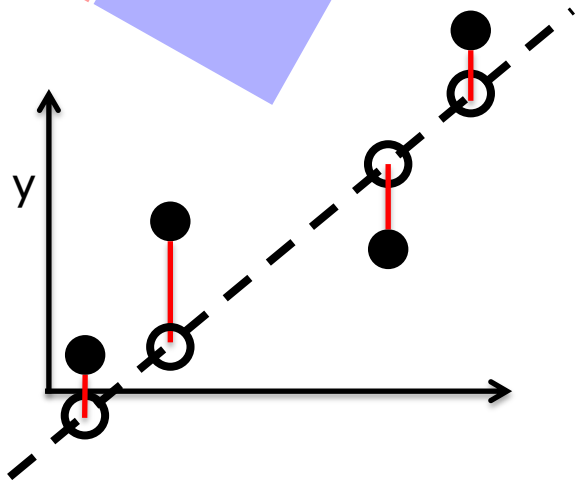
- Sigmoid/Logistic function
  - $\theta(z) = \frac{1}{1+e^{-z}} = \frac{e^z}{1+e^z}$
- Sigmoid:
  - Shape like an ”s”.
- Range of sigmoid:
  - For $z \to -\infty$ have $\theta(z) \to \frac{1}{1+\infty} = 0$
  - For $z \to \infty$ have $\theta(z) \to \frac{1}{1+0} = 1$
- Many reasons for choosing sigmoid function. We won’t cover why. Nice thing: Differentiabel.



Linear Models

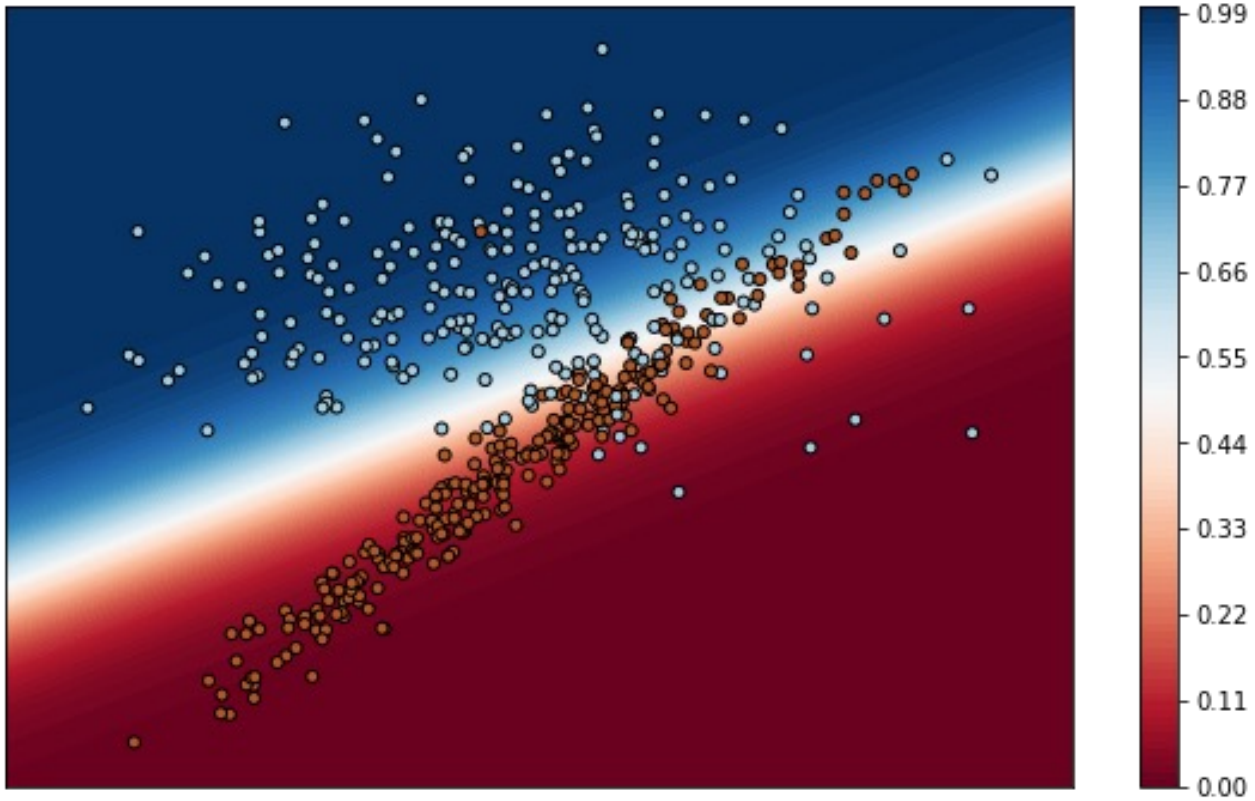
- Classification:
  - Predict(x)=sign($w^T x$)
- Regression:
  - Predict(x)=$w^T x$
- Logistic Regression:
  - Predict(x)= $\theta(w^T x)= \frac{1}{1+e^{-w^T x}}$
  - Y=[0,1]





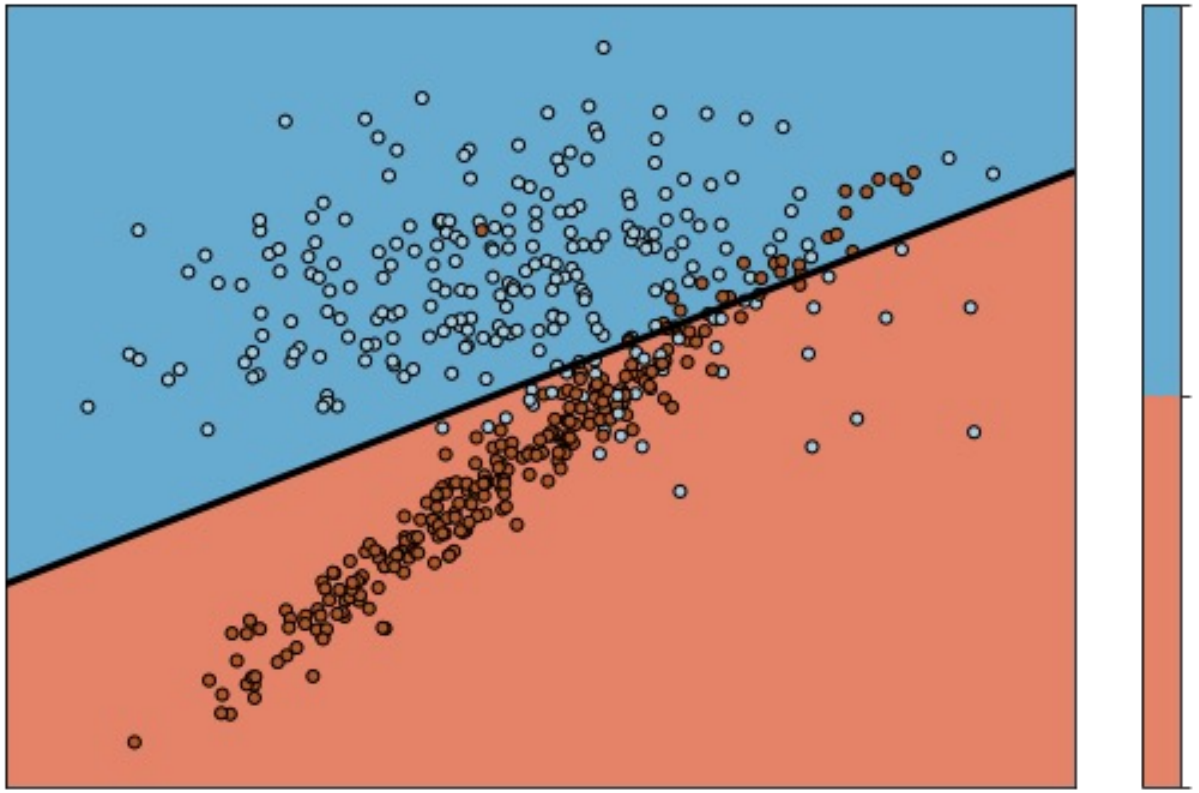
Linear Models

- Logistic Regression:
  - Predict(x)= $\theta(w^T x)= \frac{1}{1+e^{-w^T x}}$
  - Y=[0,1]



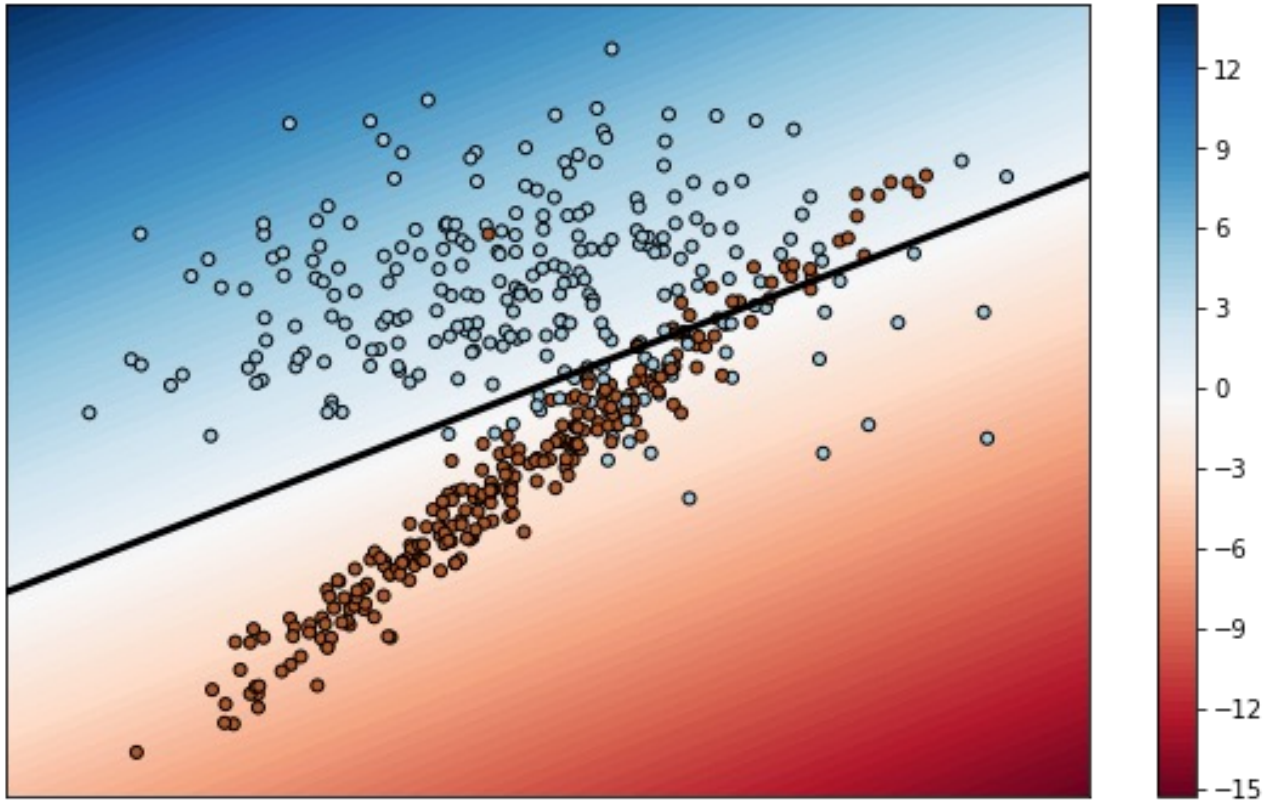
Linear Models

- Perceptron:
  - Predict(x)= sign($w^T x$)



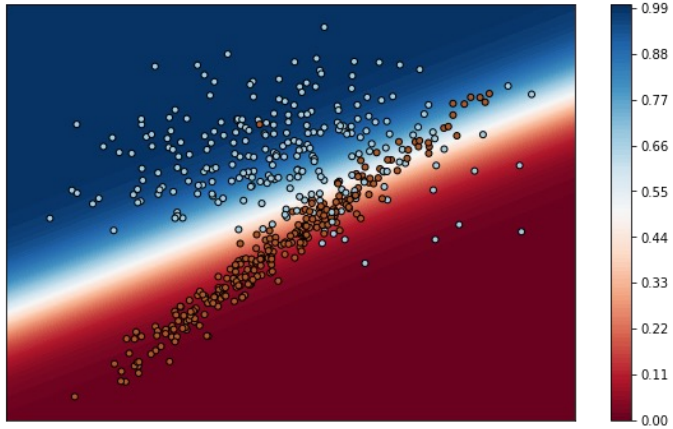
Linear Models

- Regression:
  - Predict(x)= $w^T x$



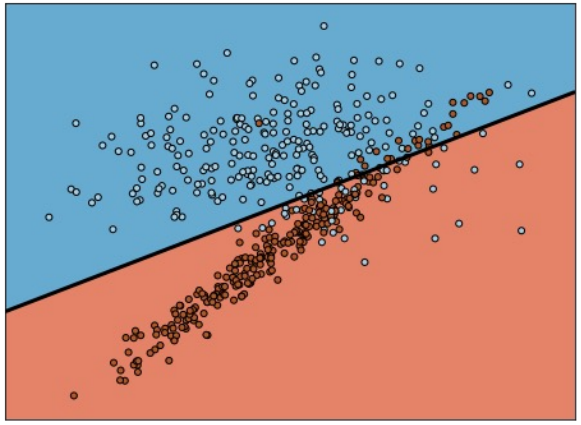
Linear Models

$\theta(w^T x) = \frac{1}{1+e^{-w^T x}}$



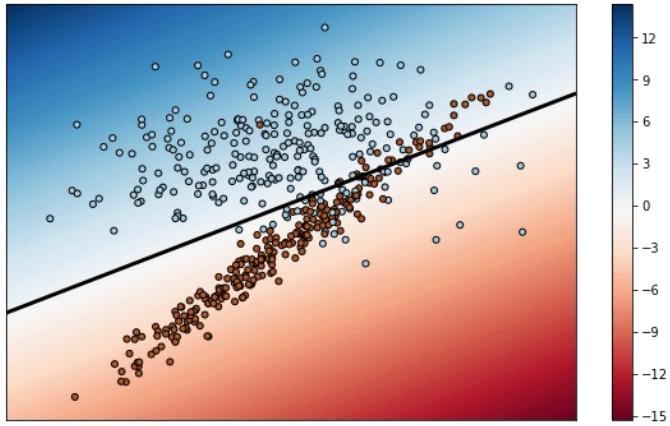
Logistic Regression

sign($w^T x$)



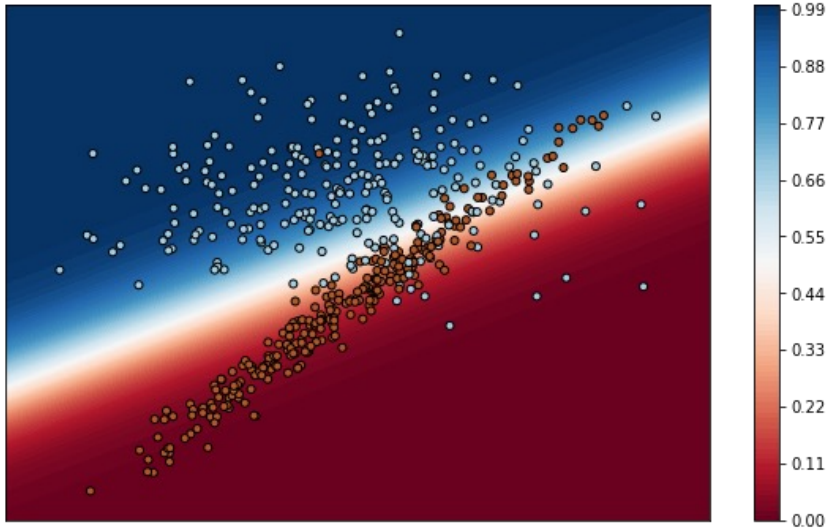
Classification

$w^T x$



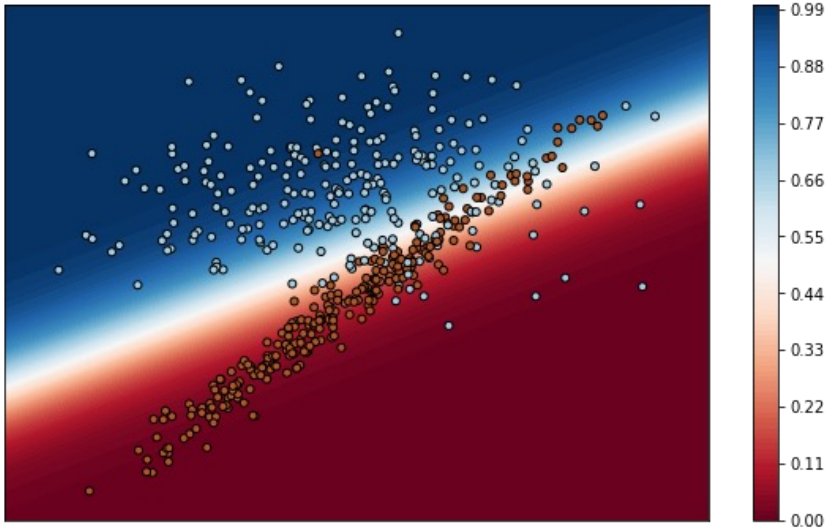
Regression

Linear Models



- Logistic Regression:
  - Predict(x)= $\theta(w^T x)= \frac{1}{1+e^{-w^T x}}$
- What loss should we minimize?
  - Classification:
    - 0-1 Loss:
      - $E_{in}(w) = \frac{1}{n}\sum_{i=1}^{n} 1_{sign(x_i^T w)\neq y_i}$
  - Regression:
    - Least squares:
      - $E_{in}(w) = \frac{1}{n}\sum_{i=1}^{n}(x_i^T w - y_i)^2$

Linear Models



- Logistic Regression:
  - Predict(x)= $\theta(w^T x)= \frac{1}{1+e^{-w^T x}}$
- Recall Intention:
  - $\theta(w^T x)$ supposed to predict P(y | x).
  - That is, we want $\theta(w^T x) \approx Pr[y = 1 \mid x]$.
    - Should estimate the probability of class 1 given features x (recall classes {-1,1}).
- Idea:
  - For any hypothesis w, can estimate ”likelihood” of training labels, denoted Pr[S | w,X] where $S=(x_1,y_1),\ldots,(x_n,y_n)$.

Maximum Likelihood

- Likelihood of training data.
  - Given data $S=(x_1,y_1),\ldots,(x_n,y_n)$.
  - Hypothesis w, i.e. $\theta(w^T x)= \frac{1}{1+e^{-w^T x}}$ should equal $Pr[y=1|x]$.
- Recall data generated as:
  - n independent samples. Draw x_i from unknown distribution D. Then draw y_i from $P(y \mid x_i)$.

- Then likelihood of seeing y_i after seeing x_i using hypothesis w:
  - If $y_i=1$: $\theta(w^T x_i)$
  - If $y_i=-1$: $1-\theta(w^T x_i)$.

  - Let us simplify a bit.

Maximum Likelihood

- Likelihood of training data.
  - Hypothesis w, i.e. $\theta(w^T x) = \frac{1}{1+e^{-w^T x}}$ should equal Pr[y=1|x].
- Then likelihood of seeing y_i after seeing x_i using hypothesis w:
  - If $y_i=1$: $\theta(w^T x_i)$
  - If $y_i=-1$: $1-\theta(w^T x_i)$.

- $1-\theta(w^T x) = 1 - \frac{1}{1+e^{-w^T x}} = \frac{1+e^{-w^T x}}{1+e^{-w^T x}} - \frac{1}{1+e^{-w^T x}} =$
- $\frac{e^{-w^T x}}{1+e^{-w^T x}} = \frac{1}{1+e^{w^T x}} = \theta(-w^T x)$.

- Likelihood: $\theta(y_i w^T x_i)$

Maximum Likelihood

- Likelihood of training data.
  - Hypothesis w, i.e. $\theta(w^T x) = \frac{1}{1+e^{-w^T x}}$ should equal Pr[y=1|x].
- Then likelihood of seeing y_i after seeing x_i using hypothesis w:
  - $\theta(y_i w^T x_i) = \frac{1}{1+e^{-y_i w^T x_i}}$
- Recall data generated as:
  - n independent samples. Draw x_i from unknown distribution D. Then draw y_i from $P(y \mid x_i)$.

Maximum Likelihood

- Likelihood of training data.
  - Hypothesis w, i.e. $\theta(w^T x) = \frac{1}{1+e^{-w^T x}}$ should equal Pr[y=1|x].
- Then likelihood of seeing y_i after seeing x_i using hypothesis w:
  - $\theta(y_i w^T x_i) = \frac{1}{1+e^{-y_i w^T x_i}}$

- Likelihood of data labels with $S=(x_1,y_1),\ldots,(x_n,y_n)$ for w:
- By independence:
  - $\Pr[S \mid w,X] = \prod_{i=1}^{n} \frac{1}{1+e^{-y_i w^T x_i}}$

Maximum Likelihood

- Likelihood of training data.
  - Hypothesis w, i.e. $\theta(w^T x) = \frac{1}{1+e^{-w^T x}}$ should equal Pr[y=1|x].
- Likelihood of data labels with $S=(x_1,y_1),\ldots,(x_n,y_n)$ for w:
  - $\Pr[S \mid w,X] = \prod_{i=1}^{n} \frac{1}{1+e^{-y_i w^T x_i}}$
- Maximum Likelihood:
  - Goal: Choose w to maximize the likelihood of training data.
  - $w^* = \text{argmax}_w \Pr[S \mid w,X]$.
  - We need to maximize $\prod_{i=1}^{n} \frac{1}{1+e^{-y_i w^T x_i}}$.

Logistic Regression

Maximum Likelihood

- Likelihood of training data.
  - Hypothesis w, i.e. $\theta(w^T x) = \frac{1}{1+e^{-w^T x}}$ should equal Pr[y=1|x].
- Likelihood of data labels with $S=(x_1,y_1),\ldots,(x_n,y_n)$ for w:
  - $\Pr[S \mid w,X] = \prod_{i=1}^{n} \frac{1}{1+e^{-y_i w^T x_i}}$

- Maximum Likelihood:
  - Goal: Choose w to maximize the likelihood of training data.
  - $w^* = \text{argmax}_w \Pr[S \mid w,X]$.
  - We need to maximize $\prod_{i=1}^{n} \frac{1}{1+e^{-y_i w^T x_i}}$.

Maximum Likelihood

- Maximum Likelihood.
  - Training data set $S=(x_1,y_1),\ldots,(x_n,y_n)$.
  - We need to maximize $\Pr[S \mid w,X]=\prod_{i=1}^{n} \frac{1}{1+e^{-y_i w^T x_i}}$.

- Two issues:
  - Normally, we deal with minimizing loss, not maximizing.
  - The product looks annoying. For other losses, we sum over all data items of a pointwise loss.
    - $\mathrm{E}_{\mathrm{in}}(\mathrm{w}) = \frac{1}{n}\sum_{i=1}^{n} 1_{sign(x_i^T w)\neq y_i}$
    - $\mathrm{E}_{\mathrm{in}}(\mathrm{w}) = \frac{1}{n}\sum_{i=1}^{n}\left(x_i^T w - y_i\right)^2$

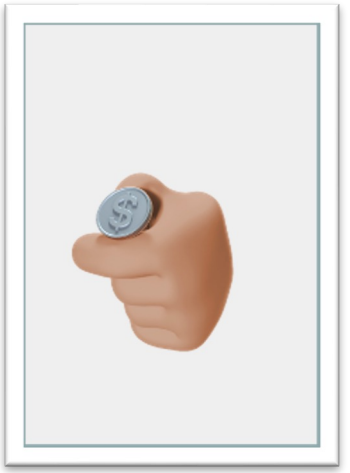
Maximum Likelihood

- Maximum Likelihood.
  - Training data set $S=(x_1,y_1),\ldots,(x_n,y_n)$.
  - We need to maximize $\Pr[S \mid w,X]=\prod_{i=1}^{n} \frac{1}{1+e^{-y_i w^T x_i}}$.

- Detour:
  - Let us look at maximum likelihood in a simpler setup.

Maximum Likelihood

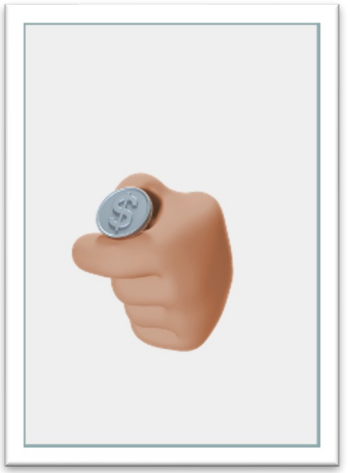
- Biased Coin:
  - Assume we have a coin with heads=1 and tails=-1.
  - The coin is biased, meaning Pr[heads]=α for some unknown α.
  - Training data is outcome of n independent flips with coin.
    - $S=x_1,x_2,x_3,x_4,\ldots$
    - 1,1,-1,1,1,-1,1,1,1,1,-1,…



- Maximum likelihood model:
  - We need to find a model, consisting of one parameter w for which we desire $w \approx \Pr[\text{heads}]=\alpha$.

Maximum Likelihood

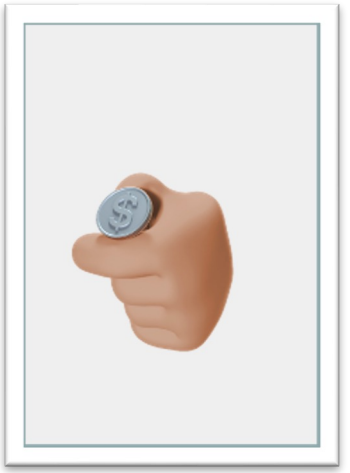
- Biased Coin:
  - Assume we have a coin with heads=1 and tails=-1.
  - The coin is biased, meaning Pr[heads]=α for some unknown α.
  - Training data is outcome of n independent flips with coin.
    - S=$x_1,x_2,x_3,x_4,\ldots$
    - Let β be fraction of heads in data.
    - Know β tends to α with more data.
    - E[β]=α.



- Model with parameter w.
- If assume w=Pr[heads], what is likelihood of data?
  - Pr[S | w].

Maximum Likelihood

- Biased Coin:
  - Assume we have a coin with heads=1 and tails=-1.
    - $S=x_1,x_2,x_3,x_4,\ldots,x_n$
  - Fraction of 1's is β.
- If assume w= Pr[heads], what is likelihood of data?



  - $\Pr[S \mid w] = \prod_{i:x_i=1} w \prod_{i:x_i=-1}(1-w)$
  - $= w^{\beta n}(1-w)^{(1-\beta)n}$.

- Maximum likelihood:
  - Use the w* that maximizes Pr[S | w*].

$$w^* = \text{argmax}_w \Pr[S \mid w]$$

Maximum Likelihood

- We need to choose w to maximize:
  - $=w^{\beta n}(1-w)^{(1-\beta)n}$.

- The products/exponentiations look annoying.
- Trick:
  - Instead minimize the negative log-likelihood:

  - -ln(Pr[S | w]) = -ln($w^{\beta n}(1-w)^{(1-\beta)n}$) = - n($\beta$ln(w) + (1-$\beta$)ln(1-w)).

Negating??

We'll see why

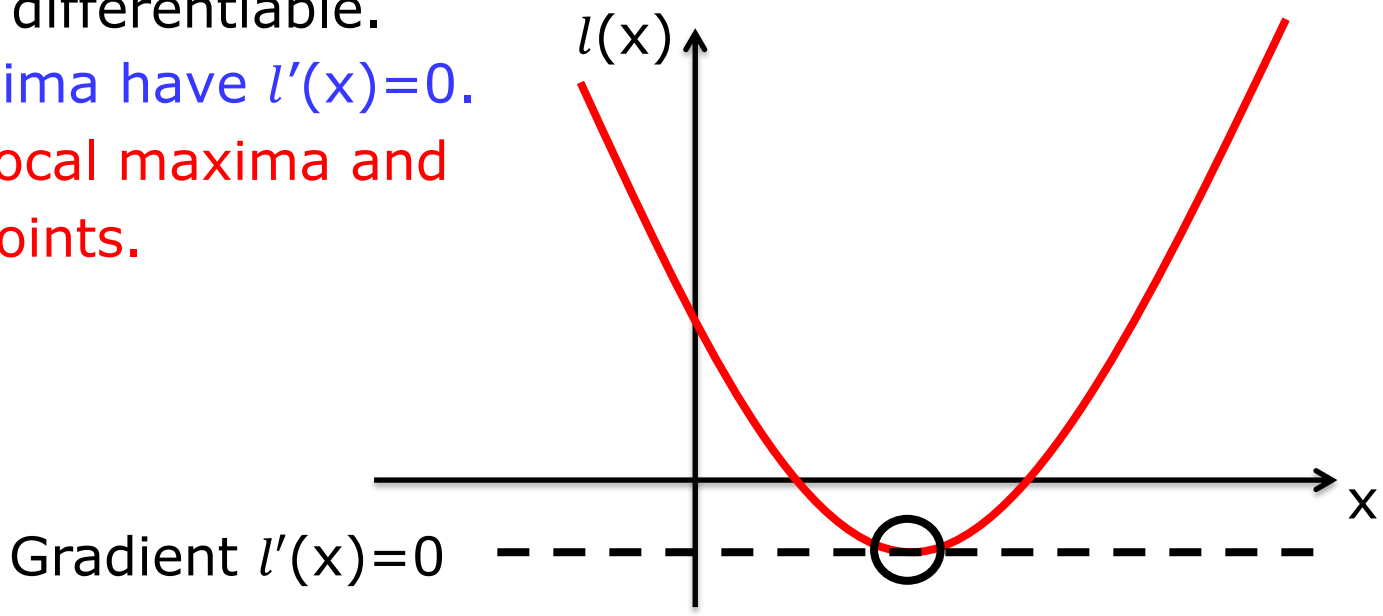
Taking ln() makes good sense.

- The -ln(x) function is strictly decreasing for x>0.
- Maximizing x (over non-negative x) is the same as minimizing –ln(x) over non-negative x.

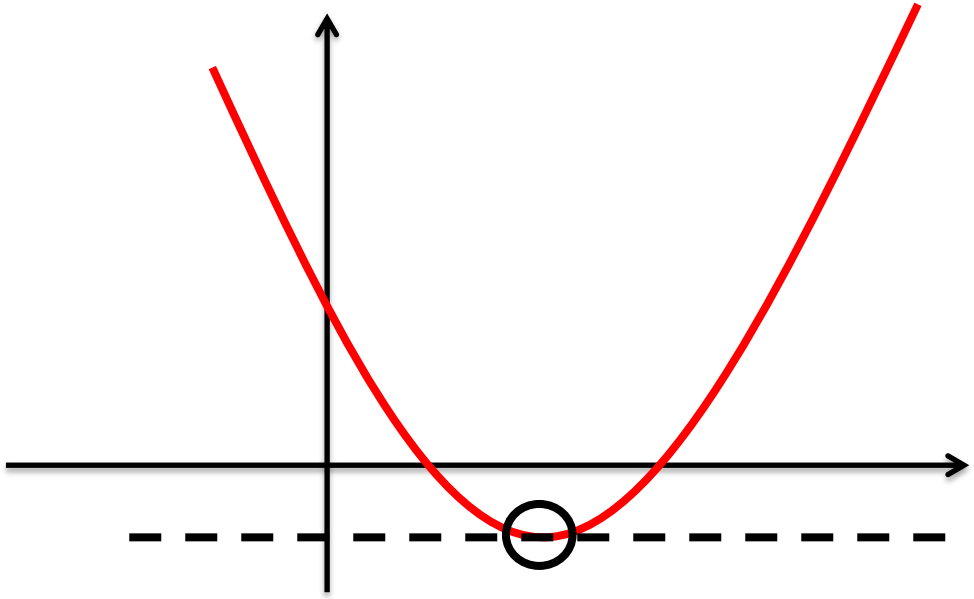
Maximum Likelihood

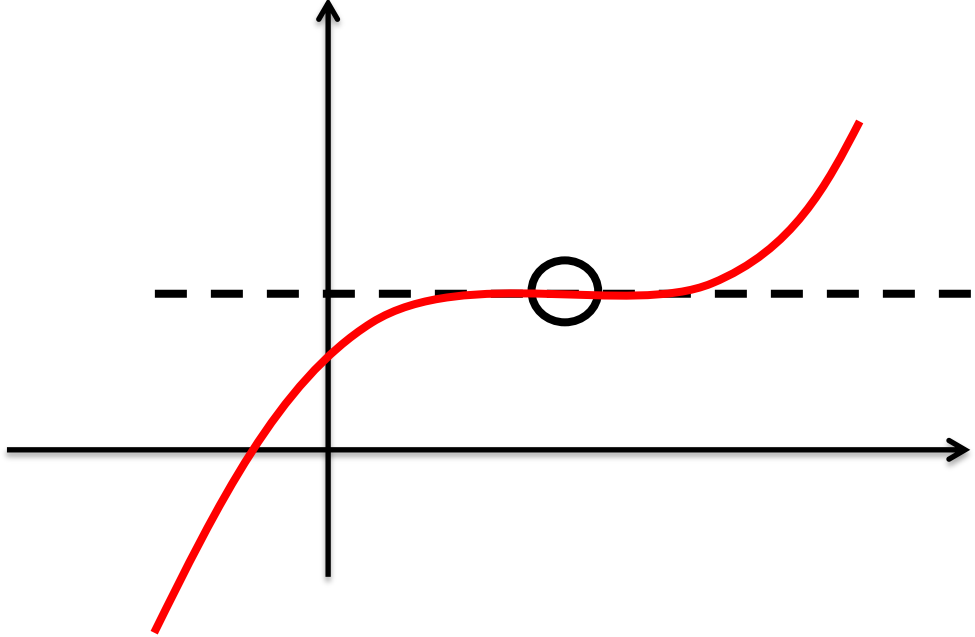
- We need to choose w to minimize:
  - $-n(\beta \ln(w) + (1-\beta)\ln(1-w))$.

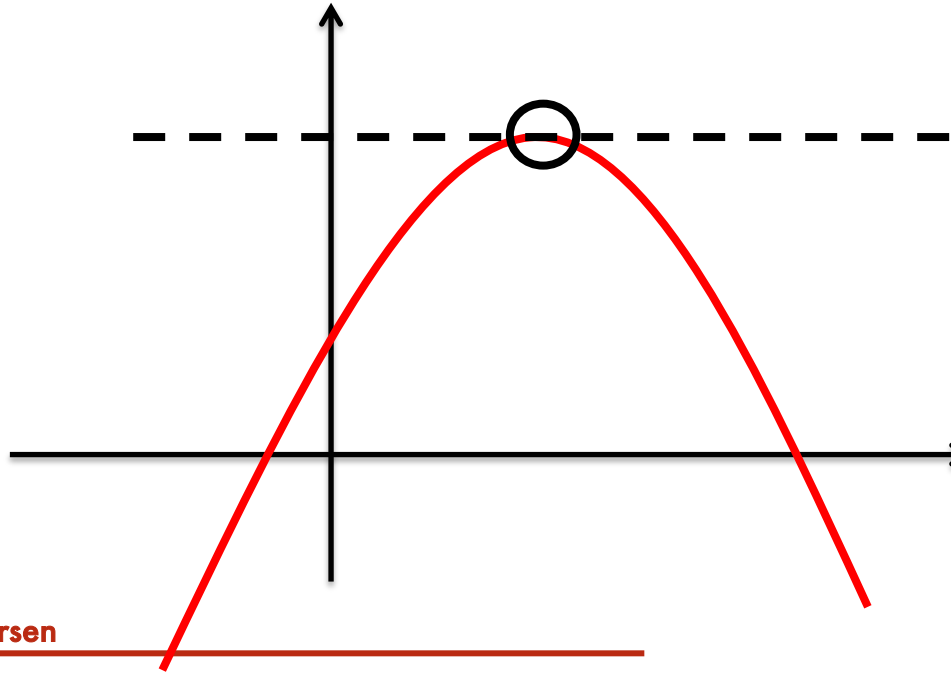
- Minimizing functions of one variable:
  - If l is a function of one variable x, i.e. from R to R.
  - Assume l differentiable.
  - Local minima have $l'(x)=0$.
  - So does local maxima and saddle points.



Local Extrema





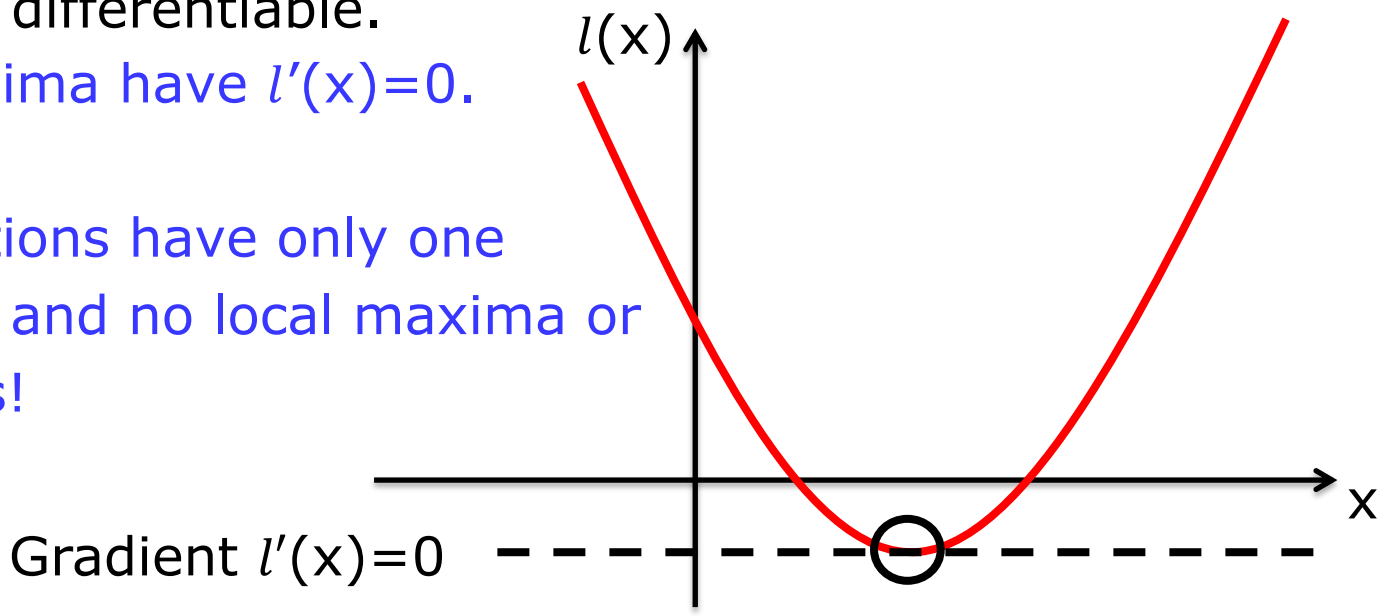


Maximum Likelihood

- We need to choose w to minimize:
  - $-n(\beta \ln(w) + (1-\beta)\ln(1-w))$.

- Minimizing functions of one variable:
  - If l is a function of one variable x, i.e. from R to R.
  - Assume l differentiable.
  - Local minima have $l'(x)=0$.

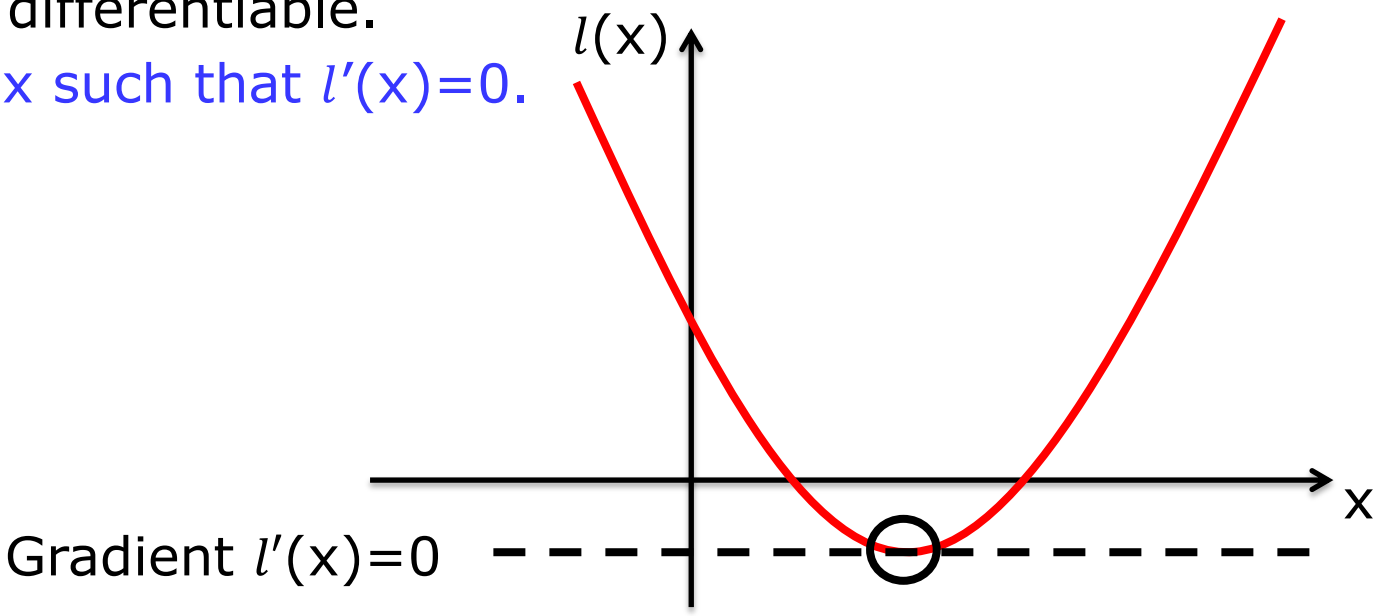
- Convex functions have only one local minima and no local maxima or saddle points!



Convex Functions

- Convex Functions:
  - Only one minima.
  - No local maxima or saddle points.

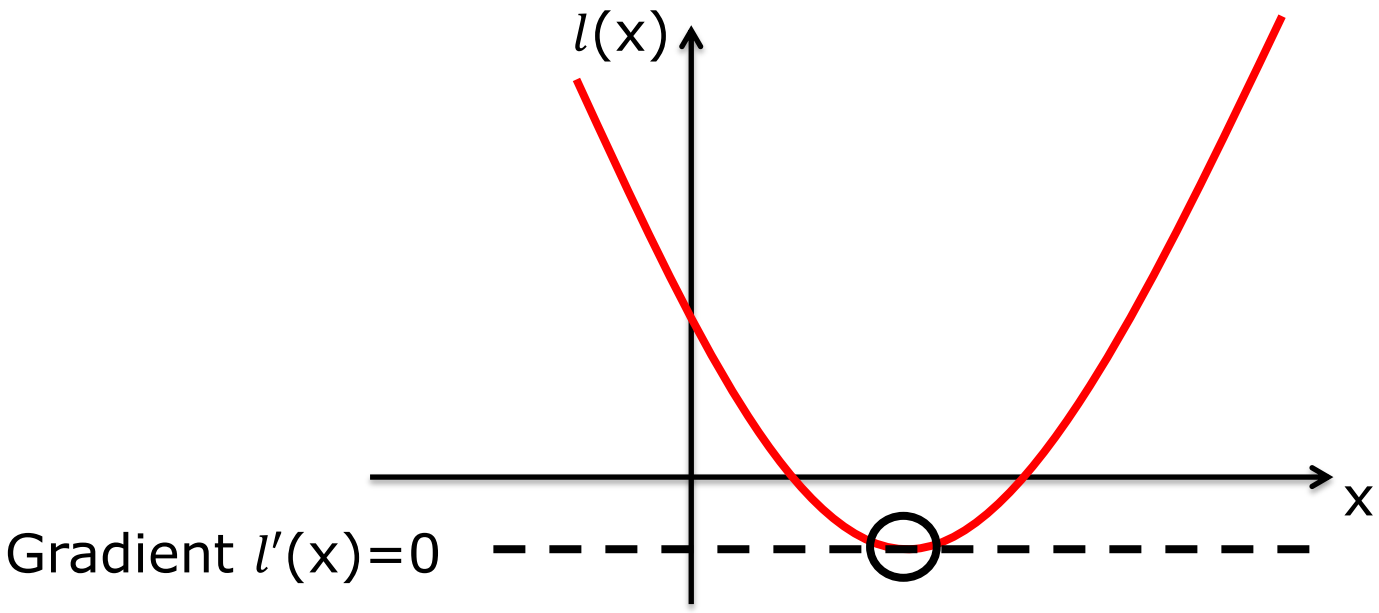
- Minimizing convex function l of one variable:
  - Assume l differentiable.
  - Compute x such that $l'(x)=0$.



Convex Functions

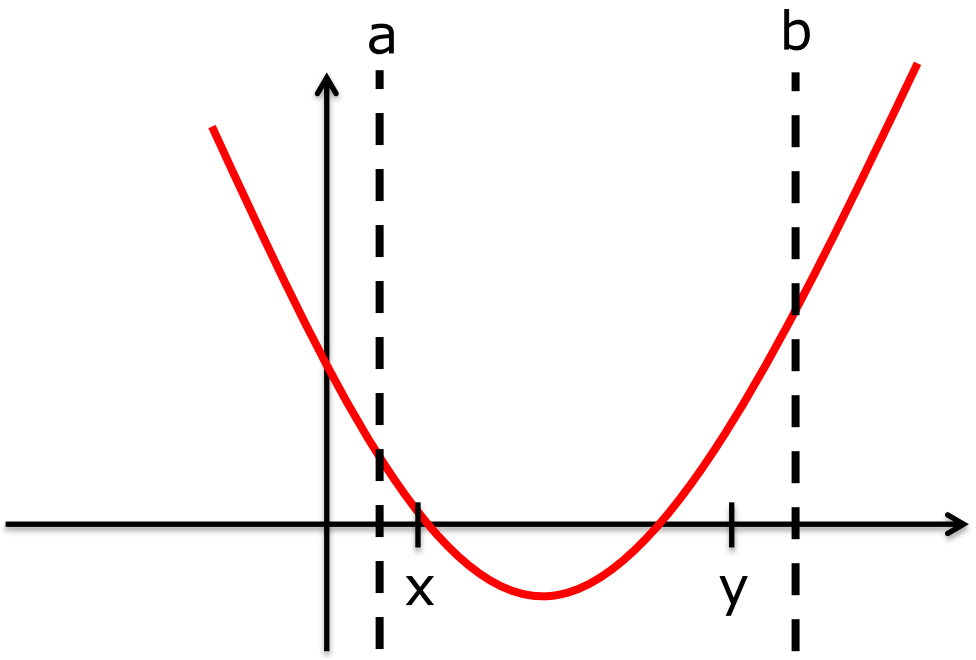
- Convex Functions:
  - Only one minima.
  - No local maxima or saddle points.

- What are convex functions??



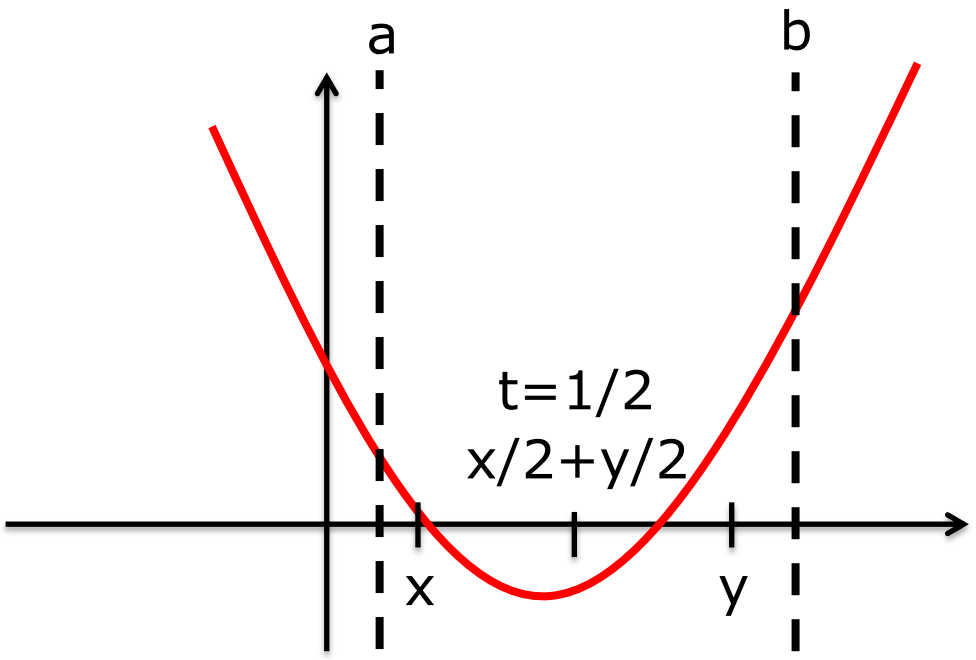
Convex Functions

- Convex Function of one variable.
  - l is convex in the interval [a,b] if:
  - For all x,y in [a,b], for all t in [0,1]:
    - $l(tx + (1-t)y) \leq tl(x) + (1-t)l(y)$.



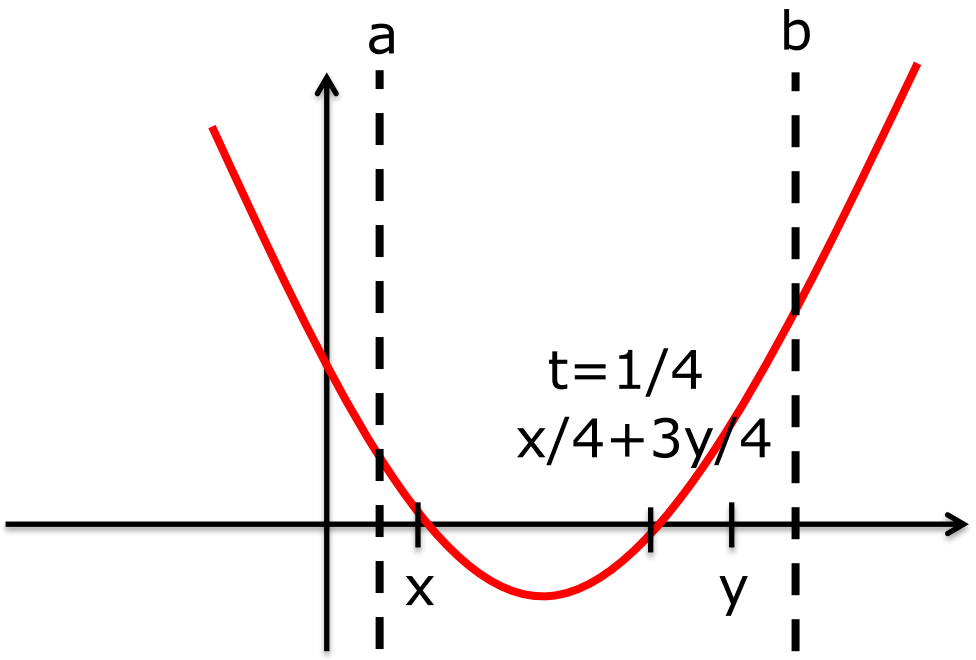
Convex Functions

- Convex Function of one variable.
  - l is convex in the interval [a,b] if:
  - For all x,y in [a,b], for all t in [0,1]:
    - $l(tx + (1-t)y) \leq tl(x) + (1-t)l(y)$.



Convex Functions

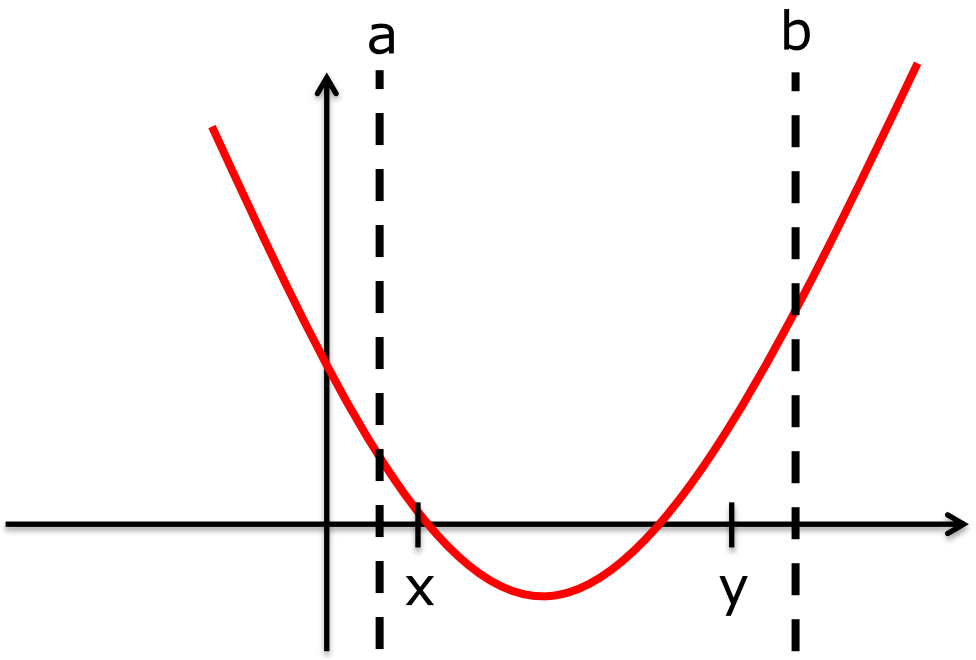
- Convex Function of one variable.
  - l is convex in the interval [a,b] if:
  - For all x,y in [a,b], for all t in [0,1]:
    - $l(tx + (1-t)y) \leq tl(x) + (1-t)l(y)$.



Convex Functions

- Convex Function of one variable.
  - l is convex in the interval [a,b] if:
  - For all x,y in [a,b], for all t in [0,1]:
    - $l(tx + (1-t)y) \leq tl(x) + (1-t)l(y)$.

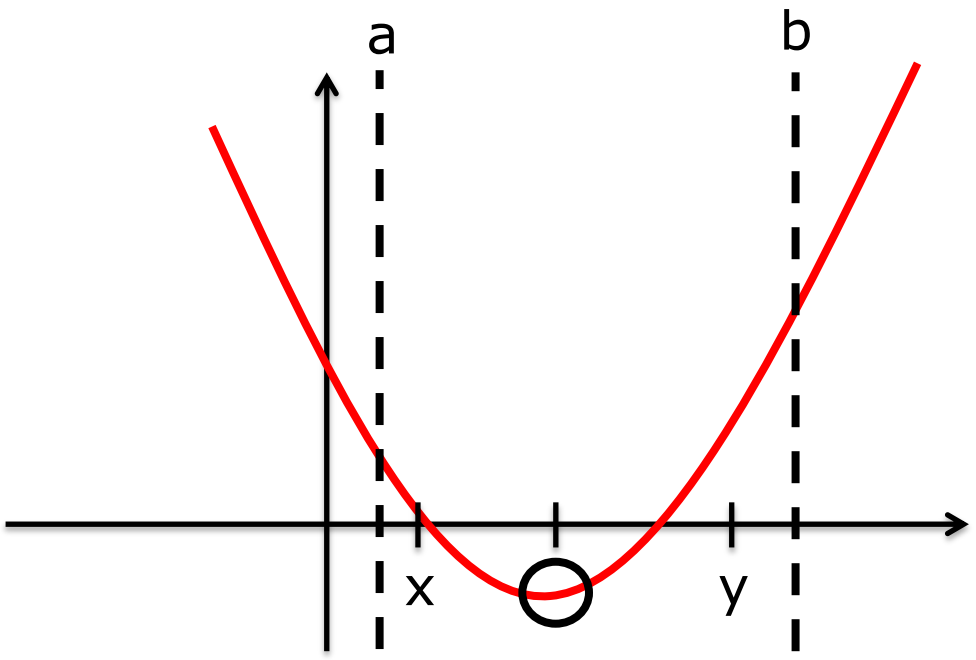
t interpolates between x and y



Convex Functions

- Convex Function of one variable.
  - l is convex in the interval [a,b] if:
  - For all x,y in [a,b], for all t in [0,1]:
    - $l(tx + (1-t)y) \leq tl(x) + (1-t)l(y)$.

t interpolates between x and y

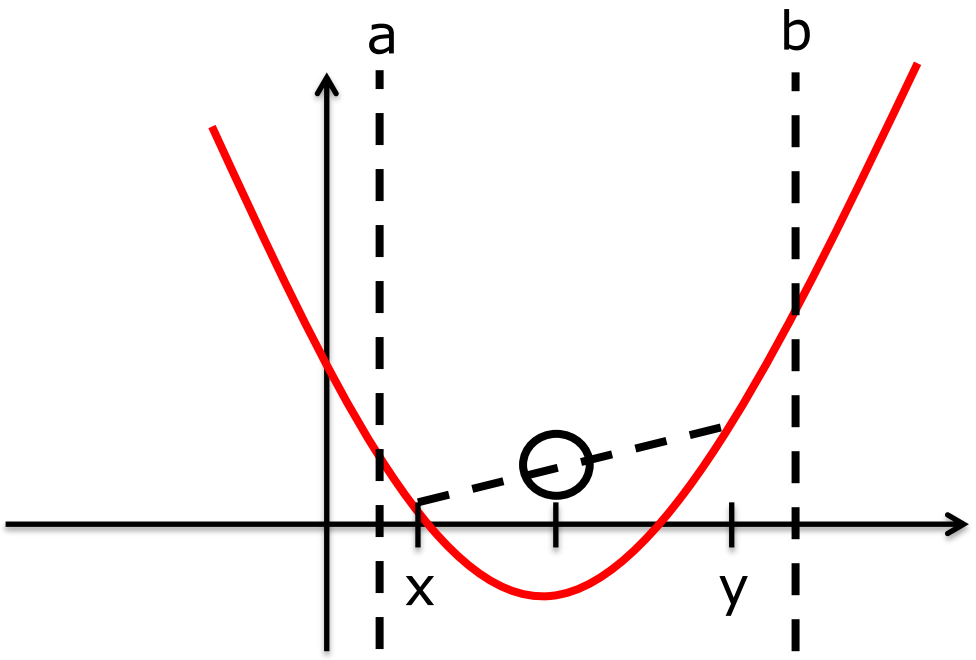


$l(tx + (1-t)y)$ is function value at interpolation point.

Convex Functions

- Convex Function of one variable.
  - l is convex in the interval [a,b] if:
  - For all x,y in [a,b], for all t in [0,1]:
    - $l(tx + (1-t)y) \leq tl(x) + (1-t)l(y)$.

t interpolates between x and y

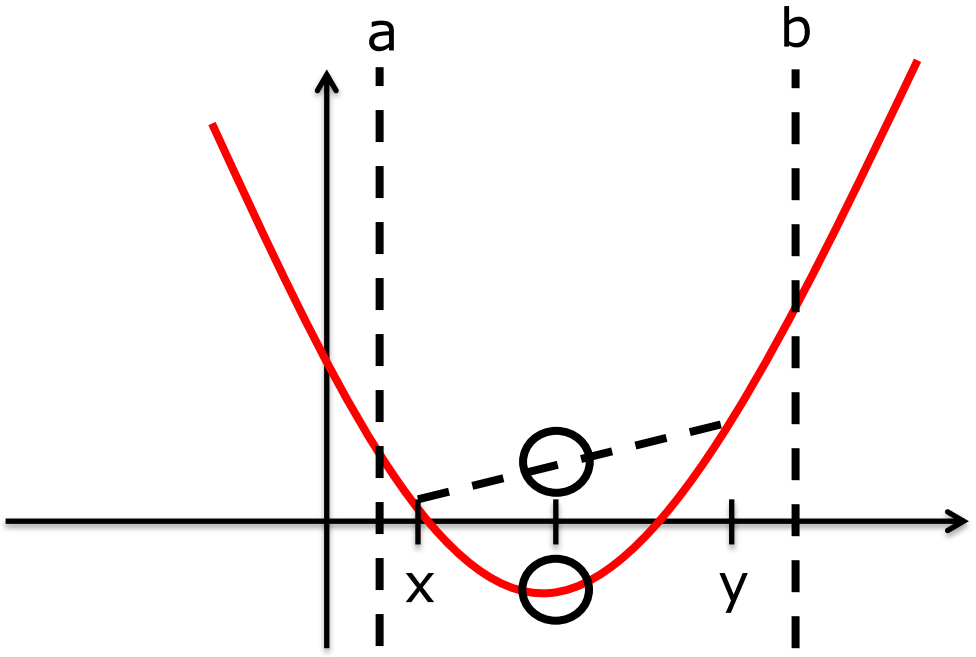


$l(tx + (1-t)y)$ is function value at interpolation point.

$tl(x)+(1-t)l(y)$ is line segment at interpolation point.

Convex Functions

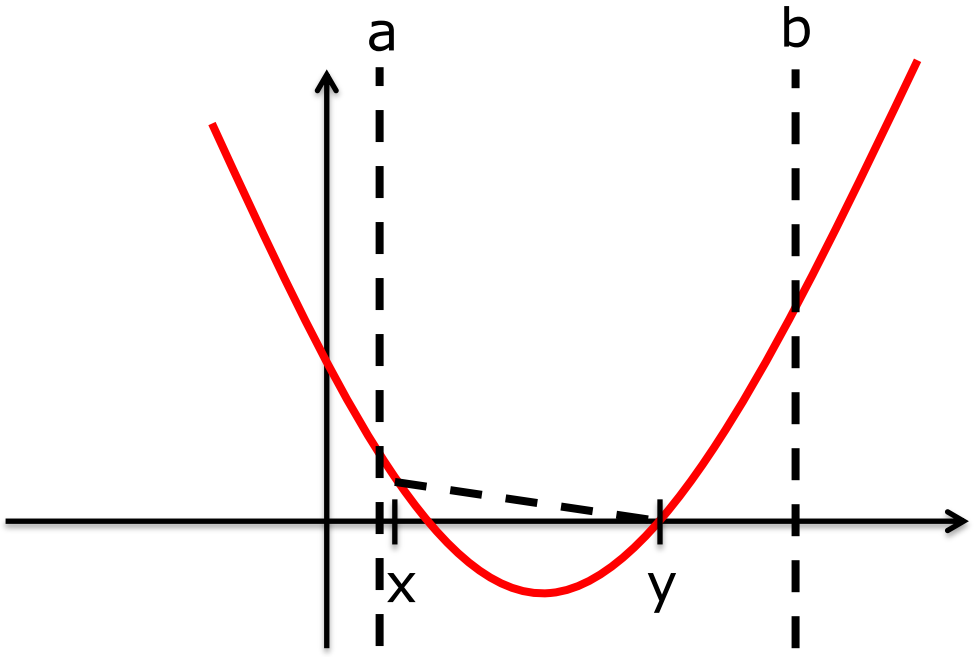
- Convex Function of one variable.
  - l is convex in the interval [a,b] if:
  - For all x,y in [a,b], for all t in [0,1]:
    - $l(tx + (1-t)y) \leq tl(x) + (1-t)l(y)$.



Convex if line segment between any two points on graph in [a,b] lie above function.

Convex Functions

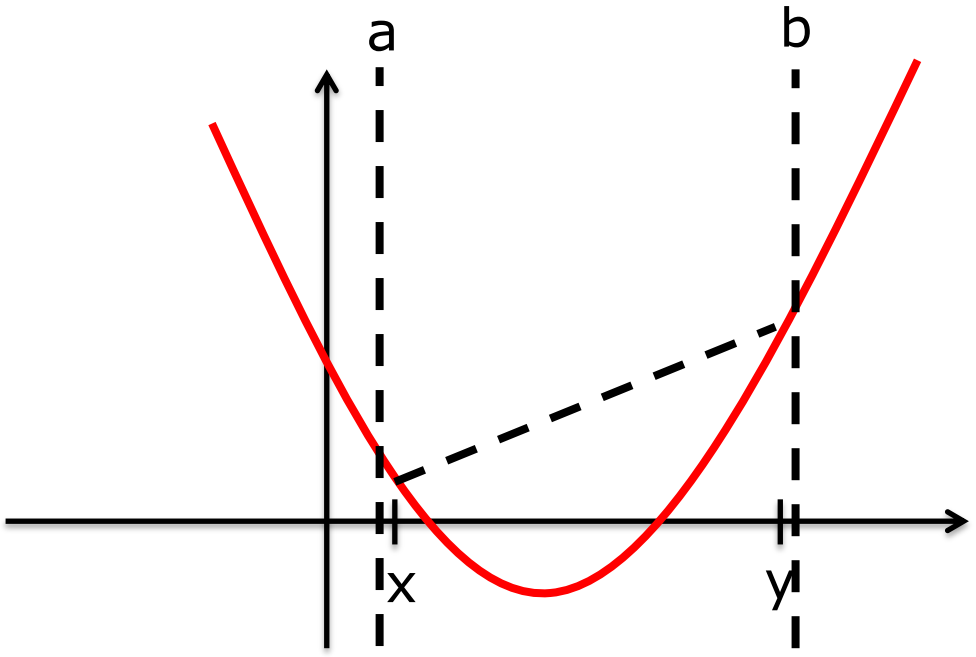
- Convex Function of one variable.
  - l is convex in the interval [a,b] if:
  - For all x,y in [a,b], for all t in [0,1]:
    - $l(tx + (1-t)y) \leq tl(x) + (1-t)l(y)$.



Convex if line segment between any two points on graph in [a,b] lie above function.

Convex Functions

- Convex Function of one variable.
  - l is convex in the interval [a,b] if:
  - For all x,y in [a,b], for all t in [0,1]:
    - $l(tx + (1-t)y) \leq tl(x) + (1-t)l(y)$.

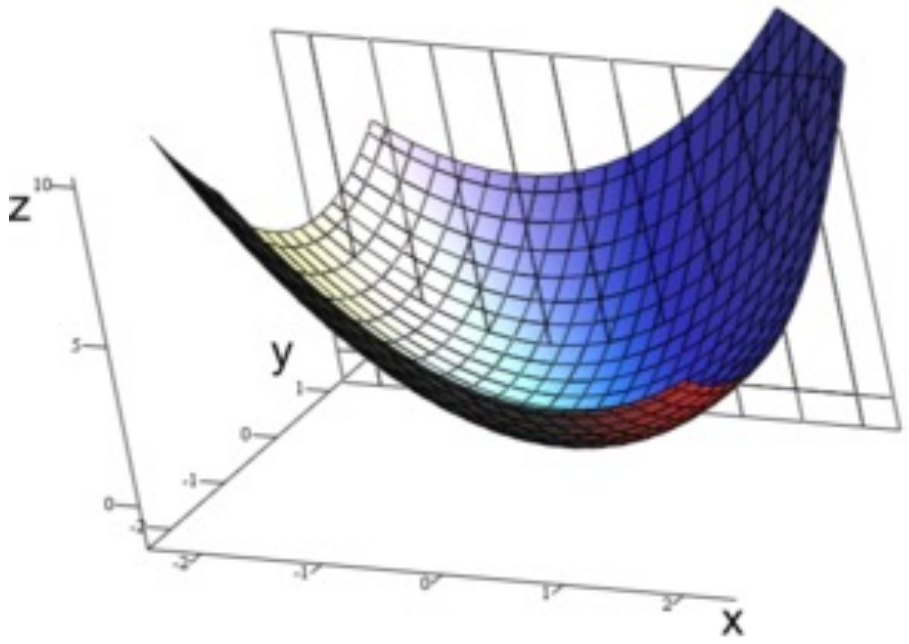


Convex if line segment between any two points on graph in [a,b] lie above function.

Convex Functions

- Convex Function of multiple variables.
  - l is convex in X, where X is convex set, if:
  - For all x,y in X, for all t in [0,1]:
    - $l(tx + (1-t)y) \leq tl(x) + (1-t)l(y)$.

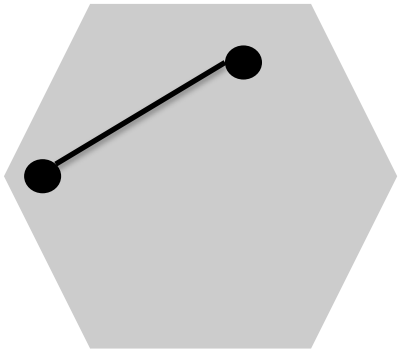
Convex if line segment between any two points on graph in X lie above function.



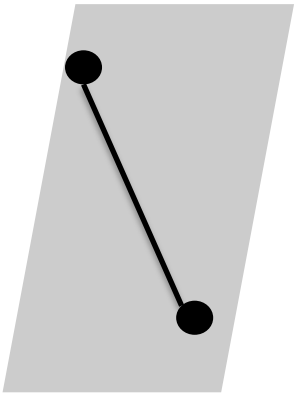
Convex Set

- Convex Sets
  - A subset X of R^d is a convex set if
    - For all x,y in X, for all t in [0,1]:
      - tx+(1-t)y is also in X.

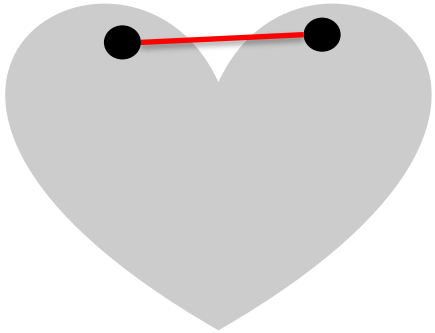
Line segment between any two points in X also lies in X.



Convex



Convex



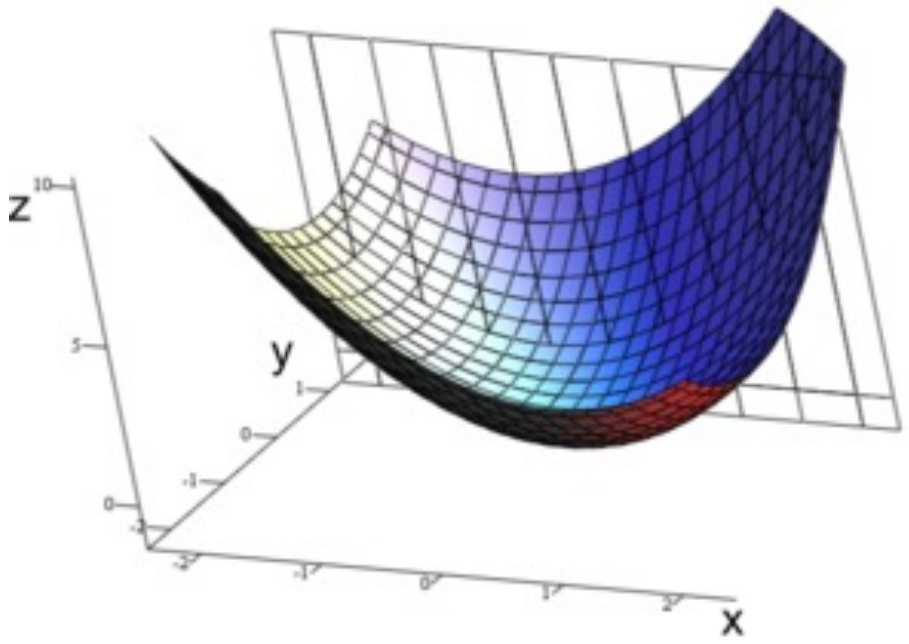
Not Convex

Convex Functions

Ensures tx+(1-t)y is in X

- Convex Function of multiple variables.
  - l is convex in X, where X is convex set, if:
  - For all x,y in X, for all t in [0,1]:
    - $l(tx + (1-t)y) \leq tl(x) + (1-t)l(y)$.

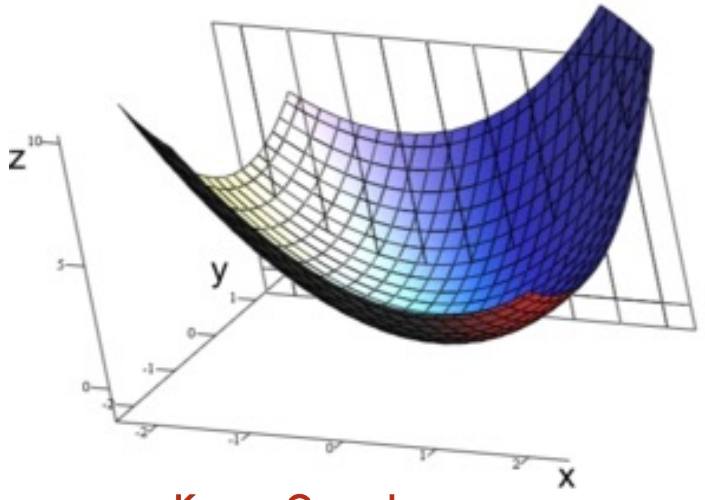
Convex function if line segment between any two points on graph in X lie above function.



Convex Functions

- Convex Functions:
  - Only one minima.
  - No local maxima or saddle points.

- Minimizing convex function l:
  - Assume l differentiable.
  - Compute x such that gradient vector $\nabla l(x)=0$.



Convex Functions

- Testing for Convexity
  - l is convex in X, where X is convex set, if:
  - For all x,y in X, for all t in [0,1]:
    - $l(\text{tx} + (1\text{-t})\text{y}) \leq \text{t}l(\text{x}) + (1\text{-t})l(\text{y})$.

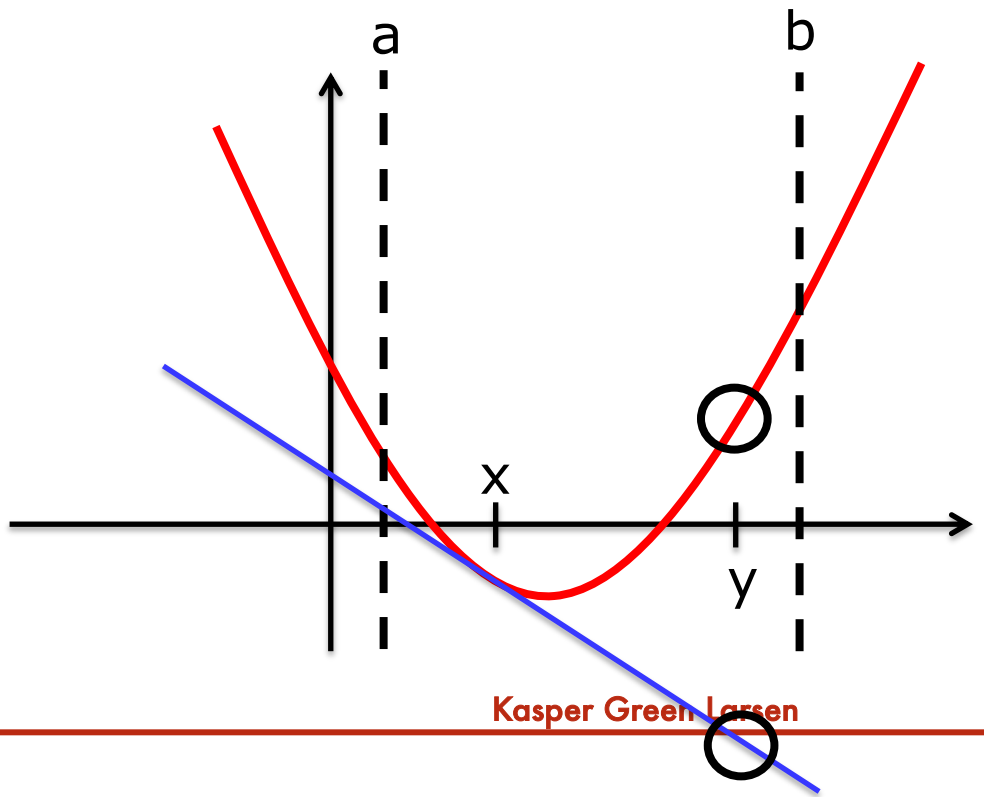
- Equivalent Definition:
  - l is convex in X, where X is convex set, if:
  - For all x,y in X:
    - $l(\text{y}) \geq l(x) + \nabla l(x)^T(y - x)$

Convex Functions

- Convex Function of one variable.
  - l is convex in [a,b] if:
  - For all x,y in [a,b]:
    - $l(\mathrm{y}) \geq l(x) + l'(x)(y - x)$

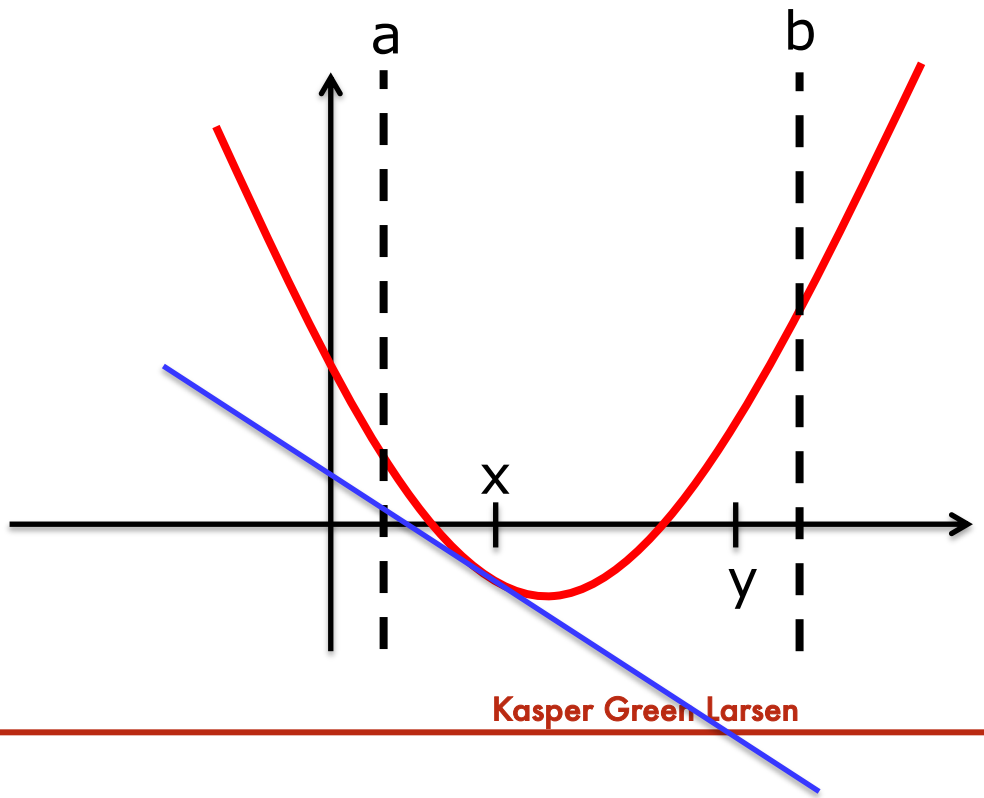
$l'(x)$ is slope of tangent

$l(x) + l'(x)\,(y - x)$ is value of tangent in x at y.



Convex Functions

- Convex Function of one variable.
  - l is convex in [a,b] if:
  - For all x,y in [a,b]:
    - $l(y) \geq l(x) + l'(x)(y - x)$



l lies above its tangents.

Convex Functions

- Testing for Convexity
  - l is convex in X, where X is convex set, if:
  - For all x,y in X, for all t in [0,1]:
    - $l(\text{tx} + (1\text{-t})\text{y}) \leq \text{t}l(\text{x}) + (1\text{-t})l(\text{y})$.

- Equivalent Definition:
  - l is convex in X, where X is convex set, if:
  - For all x,y in X:
    - $l(\text{y}) \geq l(x) + \nabla l(x)^T(y - x)$

Convex Functions

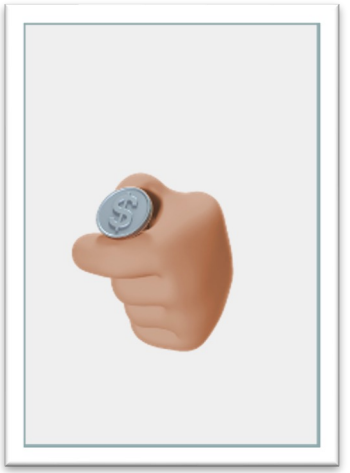
- Testing for Convexity
  - l is convex in X, where X is convex set, if:
  - For all x,y in X, for all t in [0,1]:
    - $l(\text{tx} + (1\text{-t})\text{y}) \leq \text{t}l(\text{x}) + (1\text{-t})l(\text{y})$.

- Equivalent Definition:
  - l is convex in X, where X is convex set, if:
  - For all x,y in X:
    - $l(\text{y}) \geq l(x) + \nabla l(x)^T(y - x)$

- Equivalent Definition for one variable:
  - l is convex in [a,b] if, for all x in [a,b]:
    - $l''(\text{x}) \geq 0$.

Maximum Likelihood

- Back to Maximum Likelihood
  - Assume we have a coin with heads=1 and tails=-1.
    - $S=x_1,x_2,x_3,x_4,\ldots,x_n$
  - Fraction of 1's is β.
- If assume w= Pr[heads], what is likelihood of data?



  - $\Pr[S \mid w] = \prod_{i:x_i=1} w \prod_{i:x_i=-1}(1-w)$
  - $= w^{\beta n}(1-w)^{(1-\beta)n}$.

- Maximum likelihood:
  - Use the w* that maximizes Pr[S | w*].

$$w^* = \text{argmax}_w \Pr[S \mid w]$$

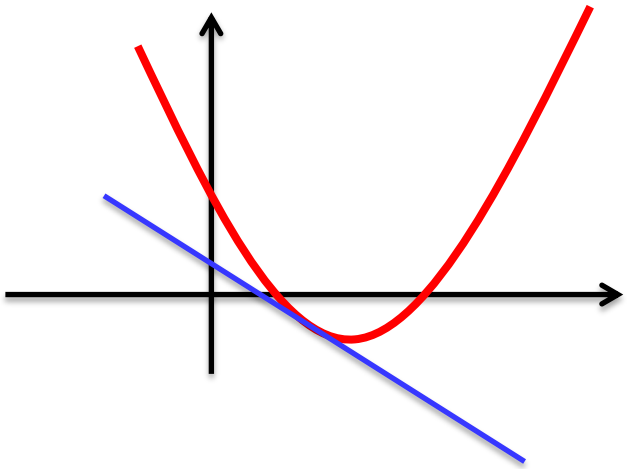
Maximum Likelihood

- Take –ln(Pr[S | w]) and minimize instead.
- We need to choose w to minimize:
  - $-\ln\left(w^{\beta n}(1-w)^{(1-\beta)n}\right) =$
  - -n(βln(w) + (1-β)ln(1-w)).

- If we can show l(w)=-n(βln(w) + (1-β)ln(1-w)) is convex, then just compute derivative and set to 0!
  - Know must choose w between 0 and 1 (a probability). Enough to show convex for w in [0,1].

Maximum Likelihood

- Take –ln(Pr[S | w]) and minimize instead.
- We need to choose w to minimize:
  - l(w)=-n(βln(w) + (1-β)ln(1-w)).

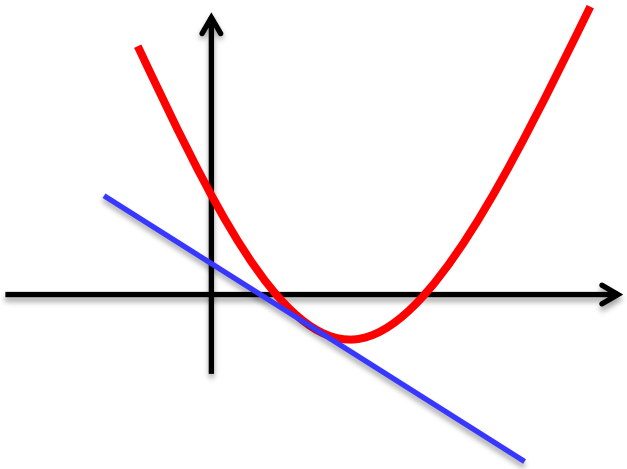


- Testing for convexity:
  - l is convex in [0,1] if, for all w in [0,1]:
    - $l''(w) \geq 0$.
- Using chain rule (for h=f(g(x)) have h'(x)=f'(g(x))g'(x)):
  - $l'(w) = -n(\frac{\beta}{w} - \frac{1-\beta}{1-w})$
  - $l''(w) = -n\left(-\frac{\beta}{w^2} - (-\frac{1-\beta}{(1-w)^2} \cdot (-1))\right)$
  - $l''(w) = n\left(\frac{\beta}{w^2} + \frac{1-\beta}{(1-w)^2}\right)$

All terms are positive for w in [0,1]!

Maximum Likelihood

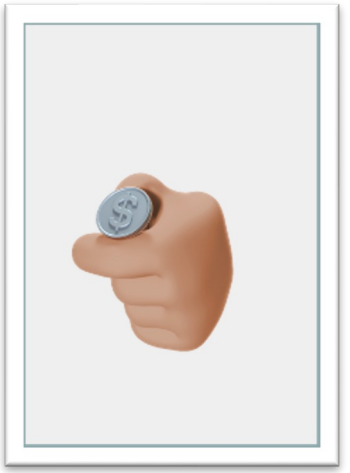
- Take –ln(Pr[S | w]) and minimize instead.
- We need to choose w to minimize:
  - $l(w) = -n(\beta \ln(w) + (1-\beta)\ln(1-w))$.
- Must set to 0:
  - $0 = l'(w) = -n(\frac{\beta}{w} - \frac{1-\beta}{1-w})$ iff
  - $0 = (\frac{\beta}{w} - \frac{1-\beta}{1-w})$ iff
  - $\frac{1-\beta}{1-w} = \frac{\beta}{w}$ iff
  - $\frac{w}{1-w} = \frac{\beta}{1-\beta}$ iff
  - $w = \beta$



Makes sense that maximum likelihood for data obtained by predicting fraction of heads in sample.

Maximum Likelihood

- Back to Maximum Likelihood
  - Assume we have a coin with heads=1 and tails=-1.
    - $S=x_1,x_2,x_3,x_4,\ldots,x_n$
  - Fraction of 1's is β.
- If assume w= Pr[heads], what is likelihood of data?



  - $\Pr[S \mid w] = \prod_{i:x_i=1} w \prod_{i:x_i=-1}(1-w)$
  - $= w^{\beta n}(1-w)^{(1-\beta)n}$.

- Maximum likelihood:
  - Use the w=β.

Maximum Likelihood

- We need to choose w to maximize:
  - $= w^{\beta n}(1-w)^{(1-\beta)n}$.

- The products/exponentiations look annoying.
- Trick:
  - Instead minimize the negative log-likelihood:

  - -ln(Pr[S | w]) = -ln($w^{\beta n}(1-w)^{(1-\beta)n}$) = - n($\beta$ln(w) + (1-$\beta$)ln(1-w)).

Negating??

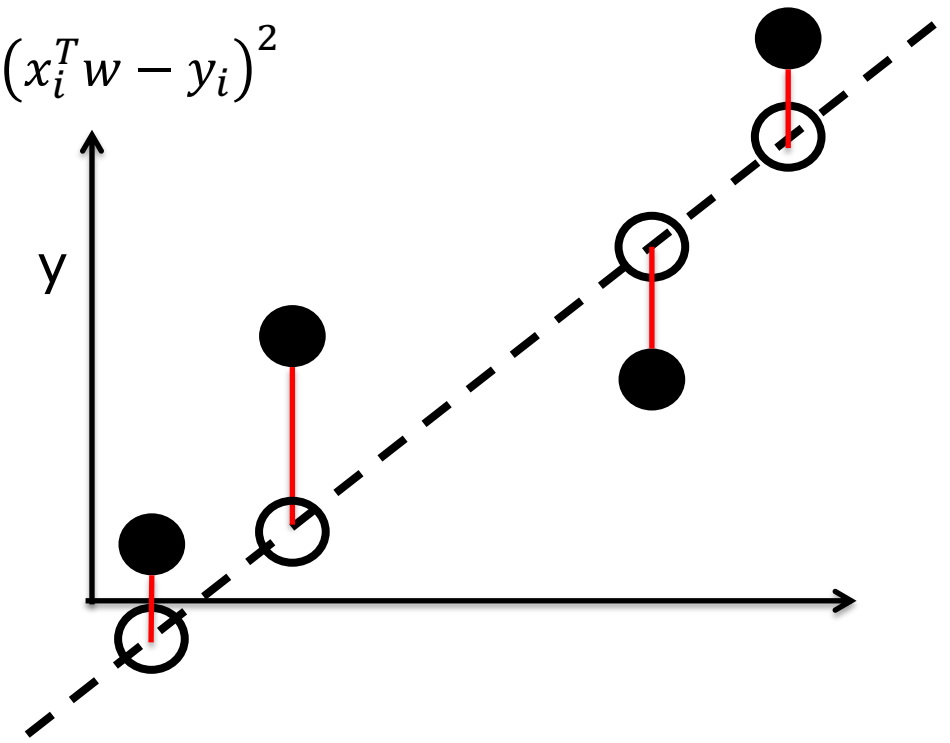
Makes convex!

Taking ln() makes good sense.

- The -ln(x) function is strictly decreasing for x>0.
- Maximizing x (over non-negative x) is the same as minimizing –ln(x) over non-negative x.

Linear Regression

- Goal: Minimize
  - $l(w) = (Xw - y)^T(Xw - y) = \sum_{i=1}^{n}(x_i^T w - y_i)^2$
  - l is a function from R^d to R.
- Strategy:
  - $\nabla_w l(w) = \begin{pmatrix} \frac{\partial}{\partial w_1} l(w) \\ \vdots \\ \frac{\partial}{\partial w_d} l(w) \end{pmatrix} = 0.$

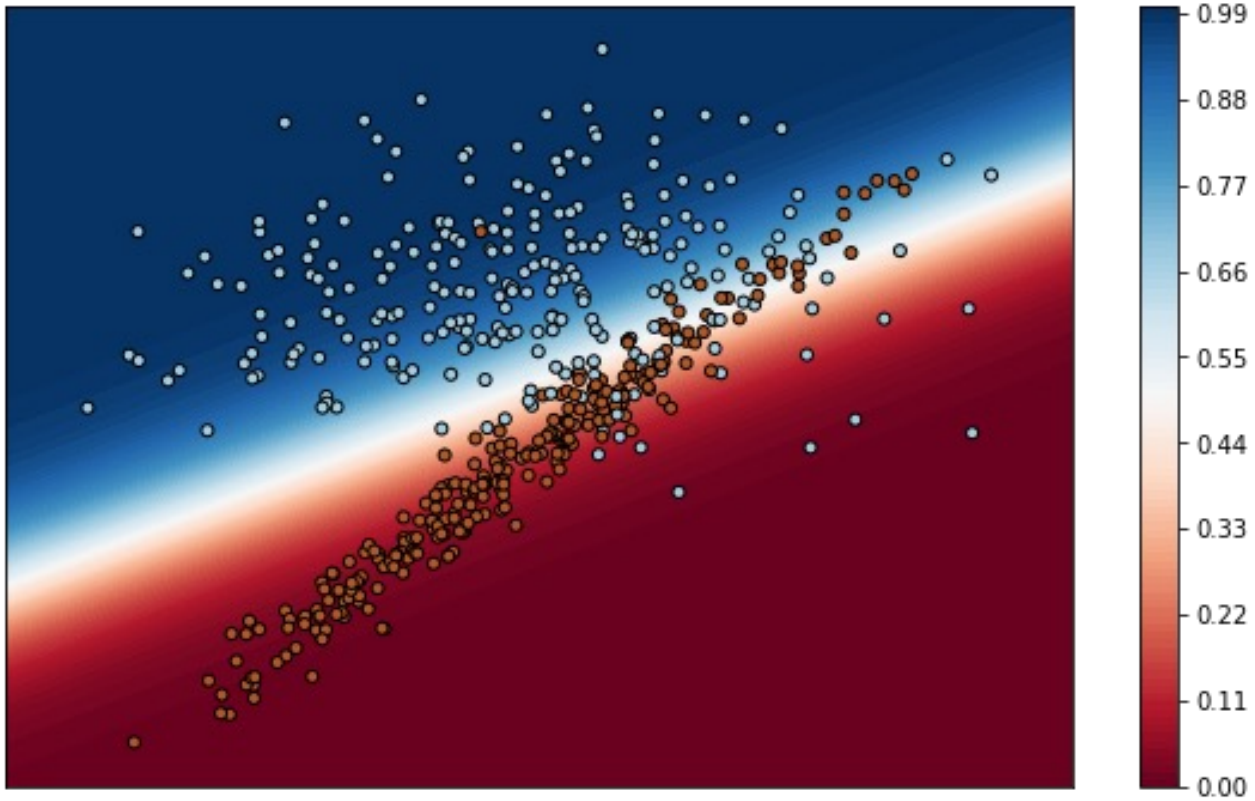


- It can be shown that linear regression loss function is convex, thus global minimum found.

Logistic Regression

Logistic Regression

- Back to Logistic Regression:
  - Predict(x)= $\theta(w^T x)= \frac{1}{1+e^{-w^T x}}$
  - Y=[0,1]



Maximum Likelihood

- Likelihood of training data.
  - Hypothesis w, i.e. $\theta(w^T x) = \frac{1}{1+e^{-w^T x}}$ should equal Pr[y=1|x].
- Likelihood of data labels with $S=(x_1,y_1),\ldots,(x_n,y_n)$ for w:
  - $\Pr[S \mid w,X] = \prod_{i=1}^{n} \frac{1}{1+e^{-y_i w^T x_i}}$
- Maximum Likelihood:
  - Goal: Choose w to maximize the likelihood of training data.
  - $w^* = \text{argmax}_w \Pr[S \mid w,X]$.
  - We need to maximize $\prod_{i=1}^{n} \frac{1}{1+e^{-y_i w^T x_i}}$.

Maximum Likelihood

- Maximum Likelihood.
  - Training data set $S=(x_1,y_1),\ldots,(x_n,y_n)$.
  - We need to maximize $\Pr[S \mid w,X]=\prod_{i=1}^{n} \frac{1}{1+e^{-y_i w^T x_i}}$.

- Two issues:
  - Normally, we deal with minimizing loss, not maximizing.
  - The product looks annoying. For other losses, we sum over all data items of a pointwise loss.
    - $\mathrm{E}_{\mathrm{in}}(\mathrm{w}) = \frac{1}{n}\sum_{i=1}^{n} 1_{sign(x_i^T w)\neq y_i}$
    - $\mathrm{E}_{\mathrm{in}}(\mathrm{w}) = \frac{1}{n}\sum_{i=1}^{n}\left(x_i^T w - y_i\right)^2$

Maximum Likelihood

- Maximum Likelihood.
  - Training data set $S=(x_1,y_1),\ldots,(x_n,y_n)$.
  - We need to maximize $\Pr[S \mid w,X]=\prod_{i=1}^{n} \frac{1}{1+e^{-y_i w^T x_i}}$.

- Trick:
  - Minimize negative log-likelihood.
    - $-\ln(\Pr[S \mid w,X]) = -\ln\left(\prod_{i=1}^{n} \frac{1}{1+e^{-y_i w^T x_i}}\right) = -\sum_{i=1}^{n} \ln\left(\frac{1}{1+e^{-y_i w^T x_i}}\right)$
    - $= \sum_{i=1}^{n} \ln\left(1+e^{-y_i w^T x_i}\right)$.
  - Same as minimizing:
    - $\frac{1}{n}\sum_{i=1}^{n} \ln\left(1+e^{-y_i w^T x_i}\right)$.

Maximum Likelihood

- Pointwise loss function:
  - Linear Classification (Perceptron):
    - $\mathrm{E_{in}}(\mathrm{w}) = \frac{1}{n}\sum_{i=1}^{n} 1_{sign(x_i^T w)\neq y_i}$
  - Linear Regression:
    - $\mathrm{E_{in}}(\mathrm{w}) = \frac{1}{n}\sum_{i=1}^{n}\left(x_i^T w - y_i\right)^2$
  - Logistic Regression:
    - $\mathrm{E_{in}}(\mathrm{w}) = \frac{1}{n}\sum_{i=1}^{n} \ln(1 + e^{-y_i w^T x_i})$

- Minimizing E_{in} for logistic regression maximizes likelihood of training data labels.

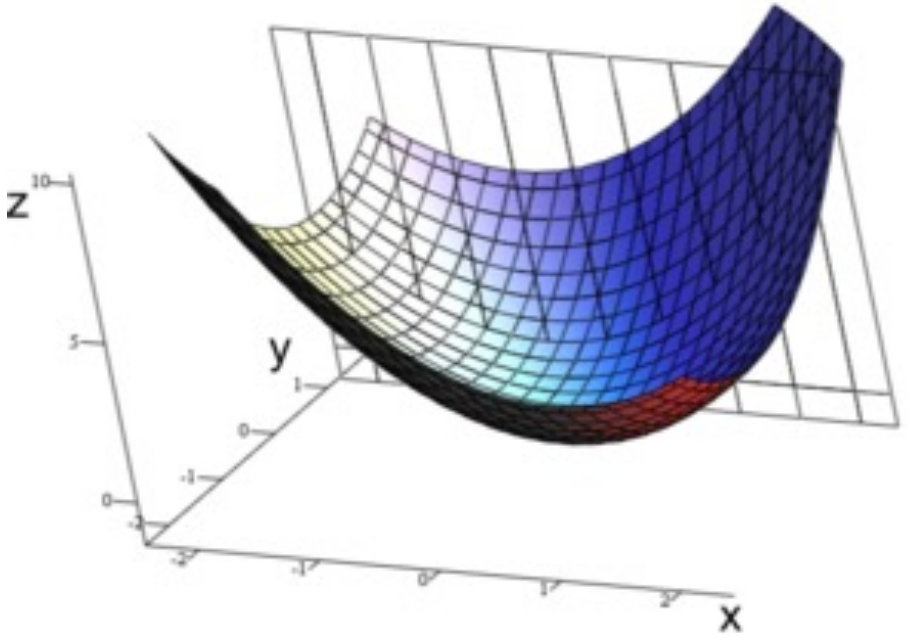
Maximum Likelihood

- Logistic Regression:
  - $\mathrm{E}_{\mathrm{in}}(\mathrm{w}) = \frac{1}{n}\sum_{i=1}^{n} \ln(1 + e^{-y_i w^T x_i})$

- Observations:
  - If $w^T x_i$ same sign as y_i, then exponent of $e^{-y_i w^T x_i}$ is negative and thus loss is small.
    - Bigger $|w^T x_i|$ makes loss smaller (more correct).
  - If $w^T x_i$ opposite sign of y_i, then exponent of $e^{-y_i w^T x_i}$ is positive and thus loss is large.
    - Bigger $|w^T x_i|$ makes loss bigger (more wrong).

Maximum Likelihood

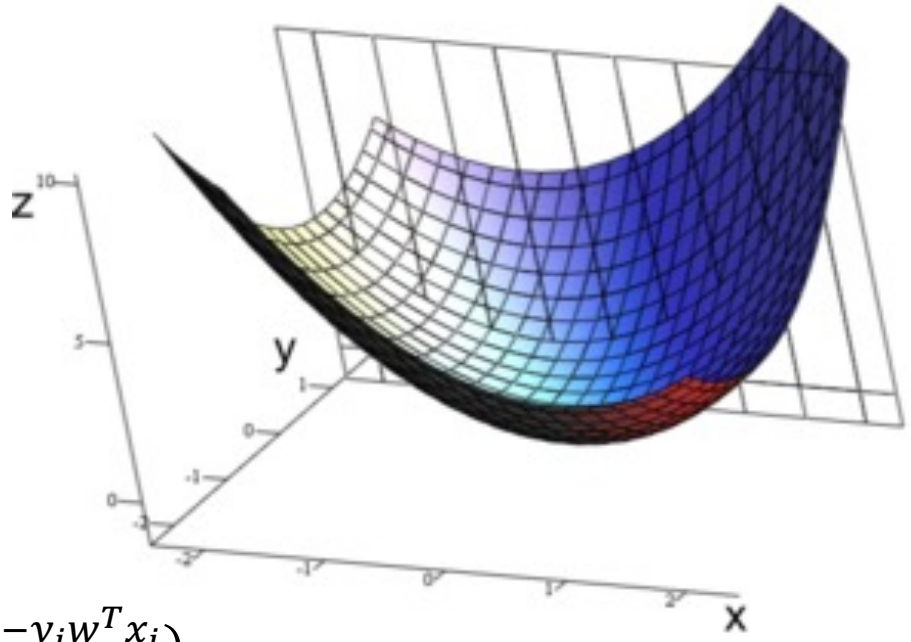
- Logistic Regression:
  - $\mathrm{E_{in}}(\mathrm{w}) = \frac{1}{n}\sum_{i=1}^{n} \ln(1 + e^{-y_i w^T x_i})$



- $E_{in}(w)$ is convex!
  - Exercise.

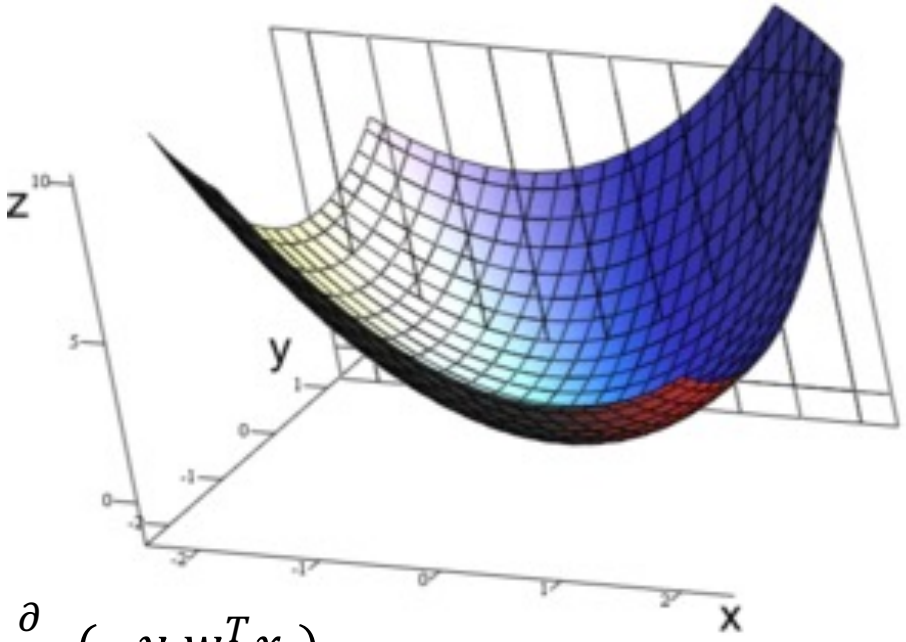
- We have a strategy for minimizing $E_{in}(w)$.
  - Compute gradient vector $\nabla \mathrm{E_{in}}(\mathrm{w})$ and set to 0.
  - Gives optimal solution since convex.

Maximum Likelihood



- Logistic Regression:
  - $\mathrm{E_{in}}(\mathrm{w}) = \frac{1}{n}\sum_{i=1}^{n} \ln(1 + e^{-y_i w^T x_i})$
- Compute gradient vector $\nabla \mathrm{E_{in}}(\mathrm{w})$.
- $\nabla \mathrm{E_{in}}(\mathrm{w})_j = \frac{\partial E_{in}(w)}{\partial w_j} = \frac{1}{n}\sum_{i=1}^{n} \frac{\partial}{\partial w_j} \ln(1 + e^{-y_i w^T x_i})$
- $= \frac{1}{n}\sum_{i=1}^{n} \frac{1}{1+e^{-y_i w^T x_i}} \cdot \frac{\partial}{\partial w_j}(1 + e^{-y_i w^T x_i})$
- $= \frac{1}{n}\sum_{i=1}^{n} \frac{1}{1+e^{-y_i w^T x_i}} \cdot \frac{\partial}{\partial w_j} e^{-y_i w^T x_i}$
- $= \frac{1}{n}\sum_{i=1}^{n} \frac{1}{1+e^{-y_i w^T x_i}} \cdot e^{-y_i w^T x_i} \cdot \frac{\partial}{\partial w_j}(-y_i w^T x_i)$

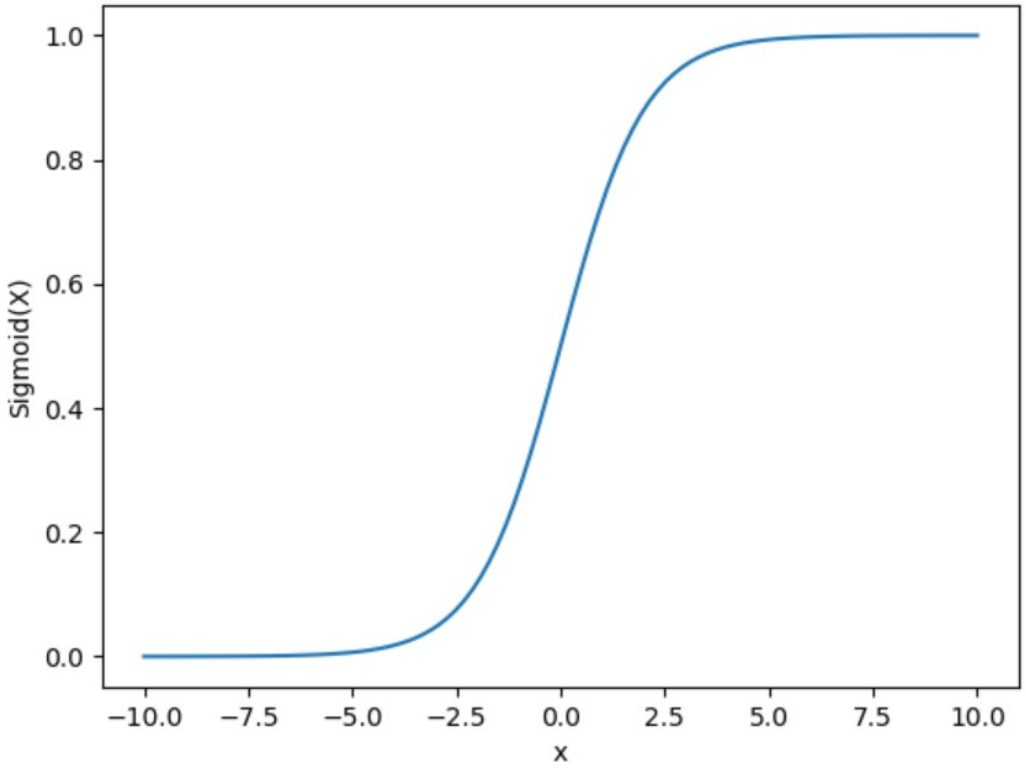
Maximum Likelihood



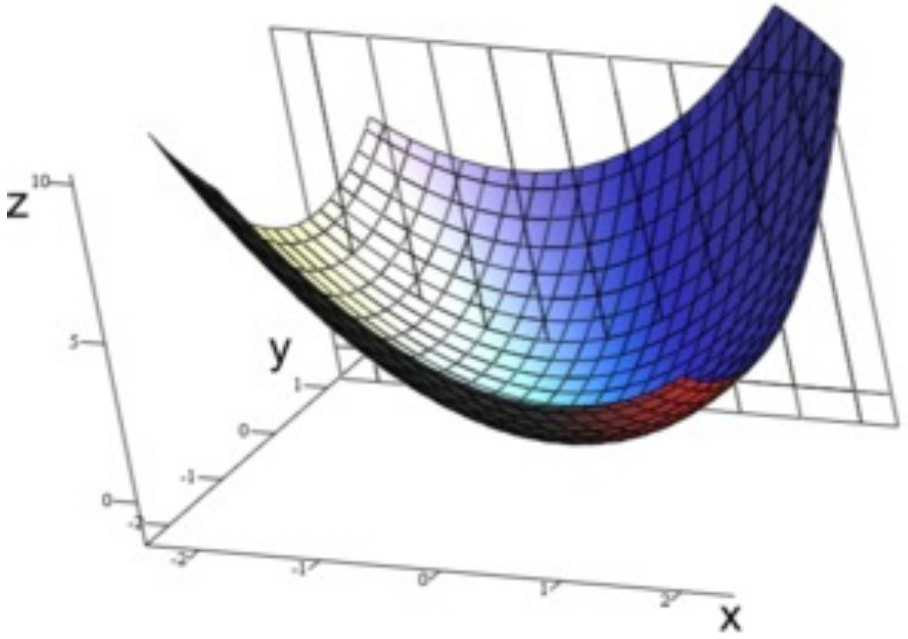
- Logistic Regression:
  - $\mathrm{E}_{\mathrm{in}}(\mathrm{w}) = \frac{1}{n}\sum_{i=1}^{n} \ln(1 + e^{-y_i w^T x_i})$
- Compute gradient vector $\nabla \mathrm{E}_{\mathrm{in}}(\mathrm{w})$.
- $\nabla \mathrm{E}_{\mathrm{in}}(\mathrm{w})_j = \frac{1}{n}\sum_{i=1}^{n} \frac{1}{1+e^{-y_i w^T x_i}} \cdot e^{-y_i w^T x_i} \cdot \frac{\partial}{\partial w_j}(-y_i w^T x_i)$
- $= \frac{1}{n}\sum_{i=1}^{n} \frac{1}{1+e^{-y_i w^T x_i}} \cdot e^{-y_i w^T x_i} \cdot (-y_i x_{i,j})$
- As a vector:
  - $\nabla \mathrm{E}_{\mathrm{in}}(w) = -\frac{1}{n}\sum_{i=1}^{n} \frac{1}{1+e^{-y_i w^T x_i}} \cdot e^{-y_i w^T x_i} \cdot y_i x_i$

Sigmoid

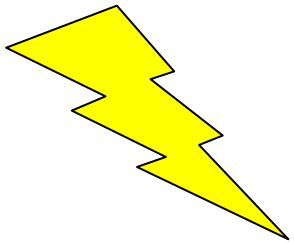
- $\nabla E_{in}(w) = -\frac{1}{n}\sum_{i=1}^{n} \frac{1}{1+e^{-y_i w^T x_i}} \cdot e^{-y_i w^T x_i} \cdot y_i x_i$
- Sigmoid/Logistic function
  - $\theta(z) = \frac{1}{1+e^{-z}} = \frac{e^z}{1+e^z}$
- $\nabla E_{in}(w) = \frac{1}{n}\sum_{i=1}^{n} -y_i x_i \, \theta(-y_i w^T x_i)$



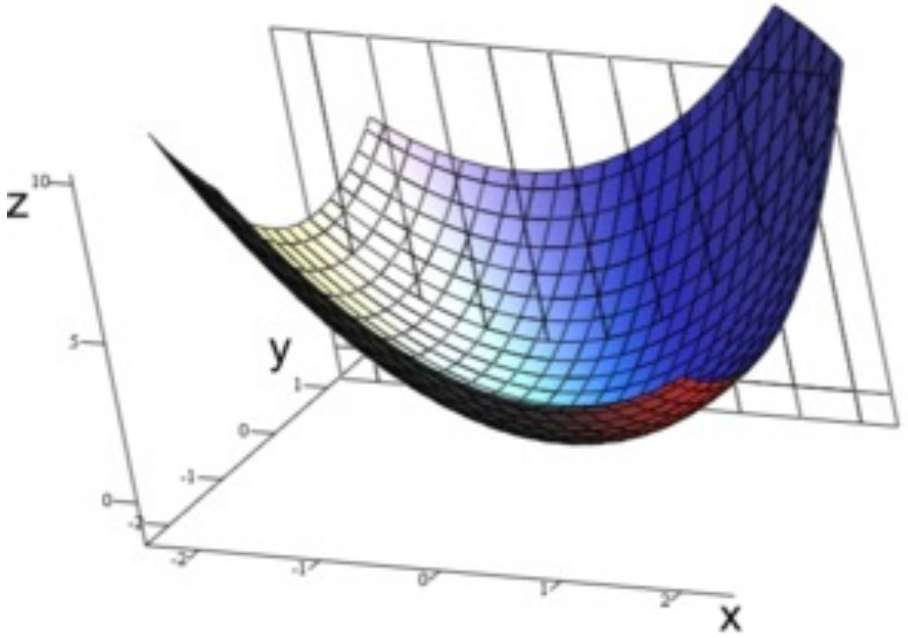
Maximum Likelihood



- Logistic Regression:
  - $\mathrm{E_{in}}(\mathrm{w}) = \frac{1}{n}\sum_{i=1}^{n} \ln(1 + e^{-y_i w^T x_i})$
- $E_{in}(w)$ is convex!
  - Exercise.
- We have a strategy for minimizing $E_{in}(w)$.
  - Compute gradient vector $\nabla \mathrm{E_{in}}(\mathrm{w})$ and set to 0.
    - $\nabla \mathrm{E_{in}}(w) = \frac{1}{n}\sum_{i=1}^{n} -y_i x_i \, \theta(-y_i w^T x_i)$
- Set to 0??
  - No analytic solution.



Gradient Descent



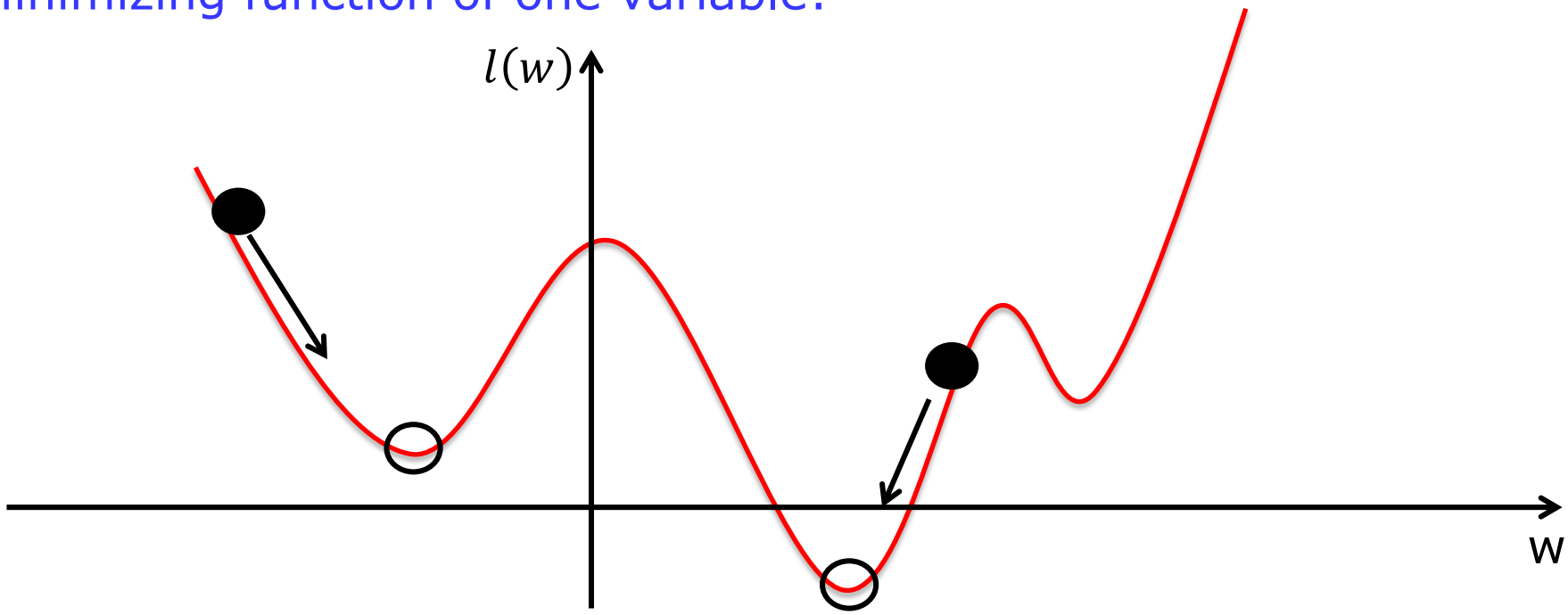
- Logistic Regression:
  - $\mathrm{E_{in}}(\mathrm{w}) = \frac{1}{n}\sum_{i=1}^{n} \ln(1 + e^{-y_i w^T x_i})$
- $E_{in}(w)$ is convex!
  - $\nabla \mathrm{E_{in}}(w) = \frac{1}{n}\sum_{i=1}^{n} -y_i x_i \, \theta(-y_i w^T x_i)$
- Cannot set to 0 analytically.

- How do we then minimize $E_{in}(w)$?
- And what can we use convexity for?

- Technique:
  - Gradient Descent.

Gradient Descent

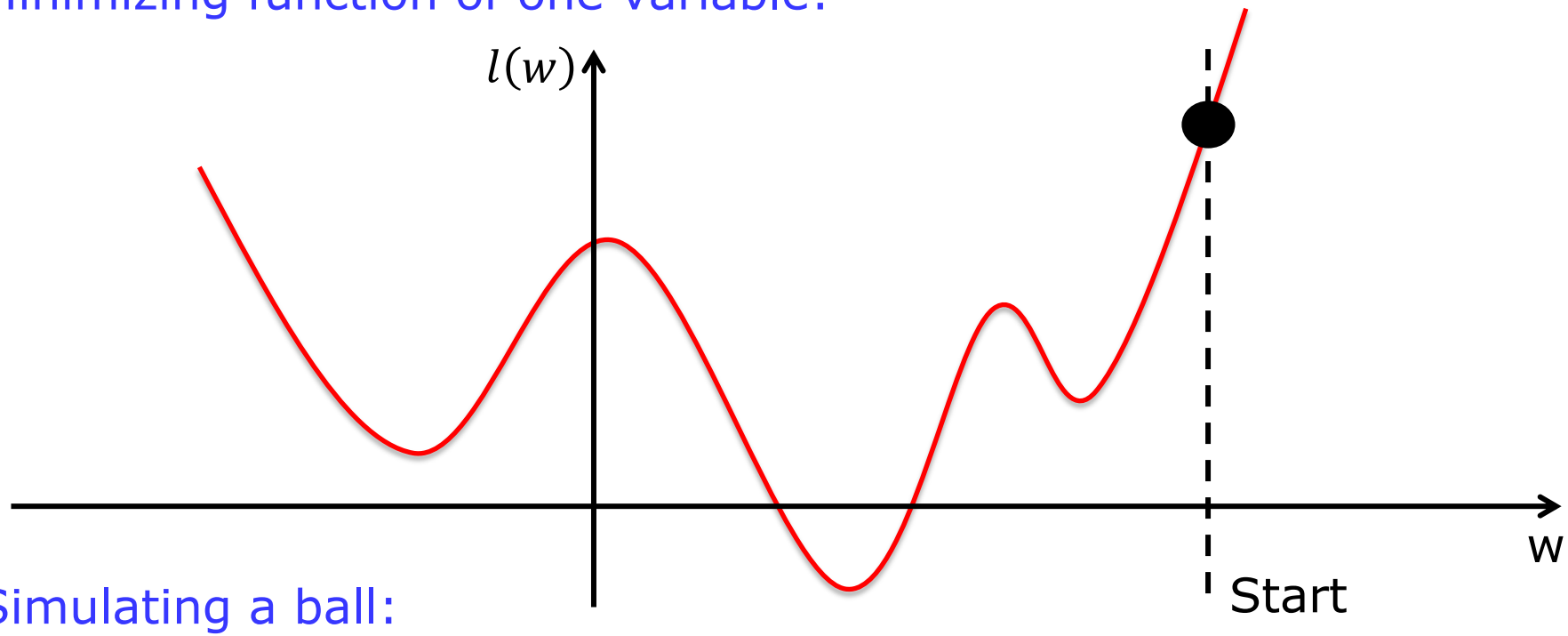
- Minimizing function of one variable:



- Conceptual idea:
  - Think of a ball rolling downhill.
  - Ends up in local minimum.

Gradient Descent

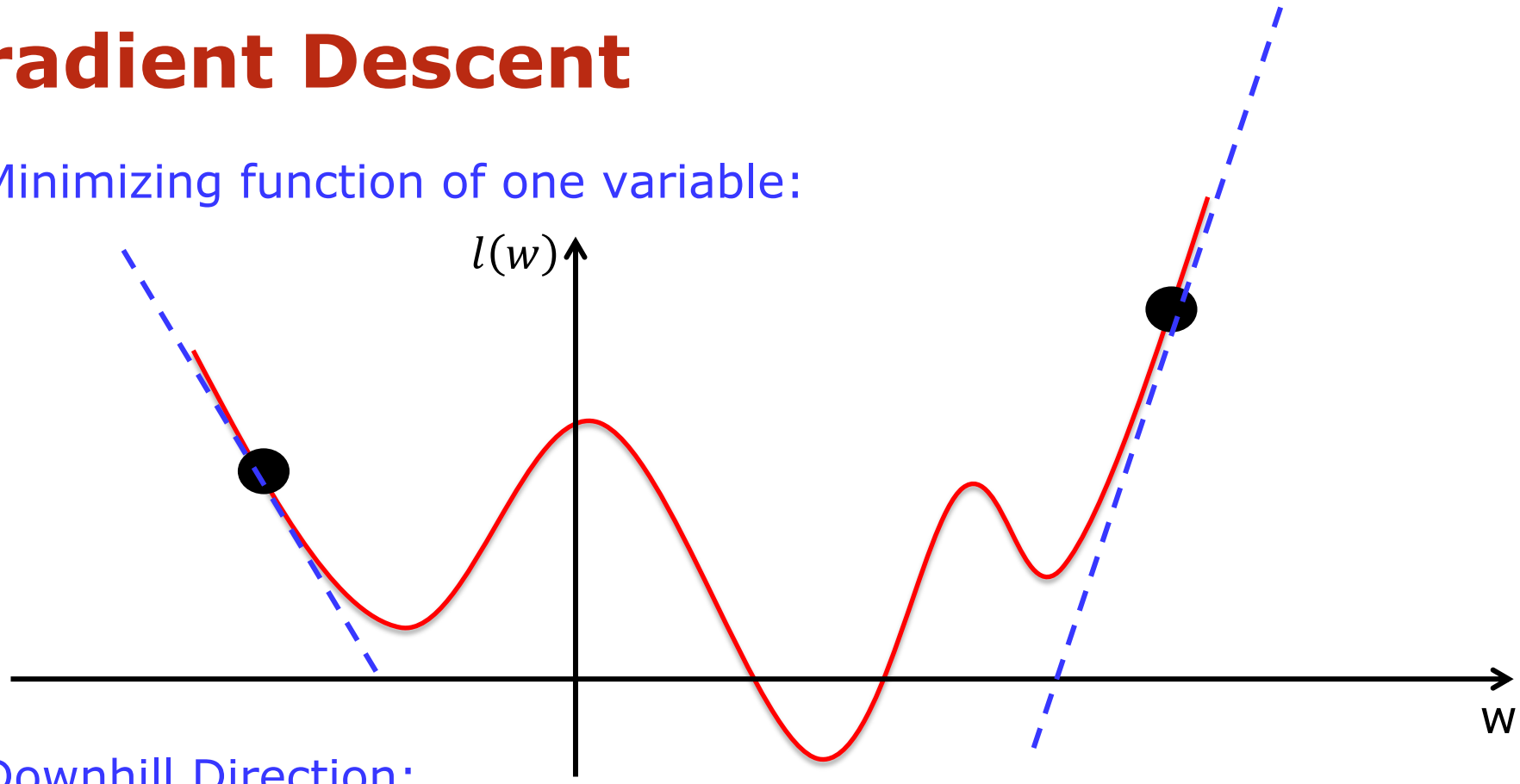
- Minimizing function of one variable:



- Simulating a ball:
  - Initialize start w somehow (random/clever).
  - Find downhill direction

Gradient Descent

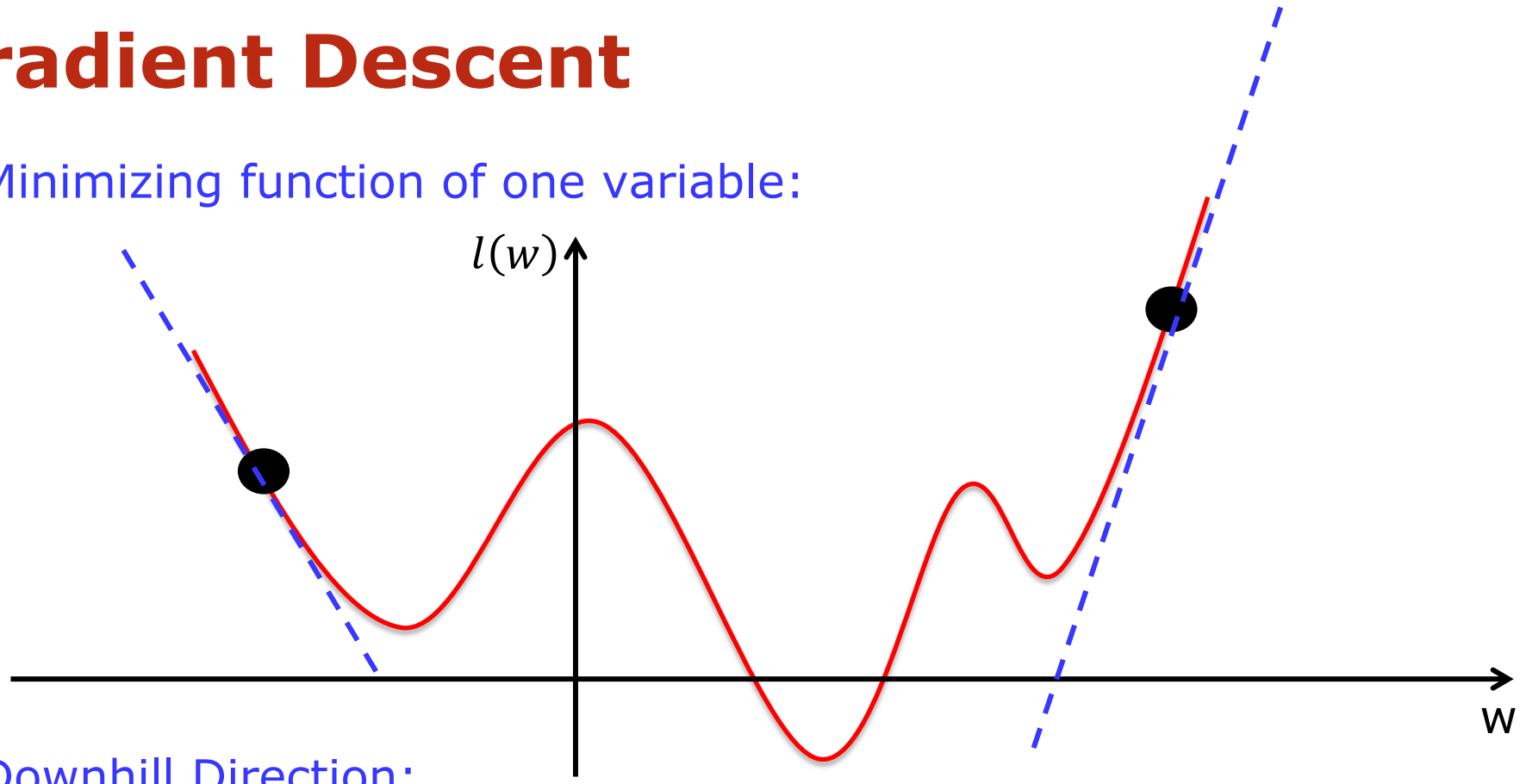
- Minimizing function of one variable:



- Downhill Direction:
  - Compute gradient $\nabla l(w)$ at w.
  - If positive, increasing at w, so ”roll” backwards.
  - If negative, decreasing at w, so ”roll” forwards.

Gradient Descent

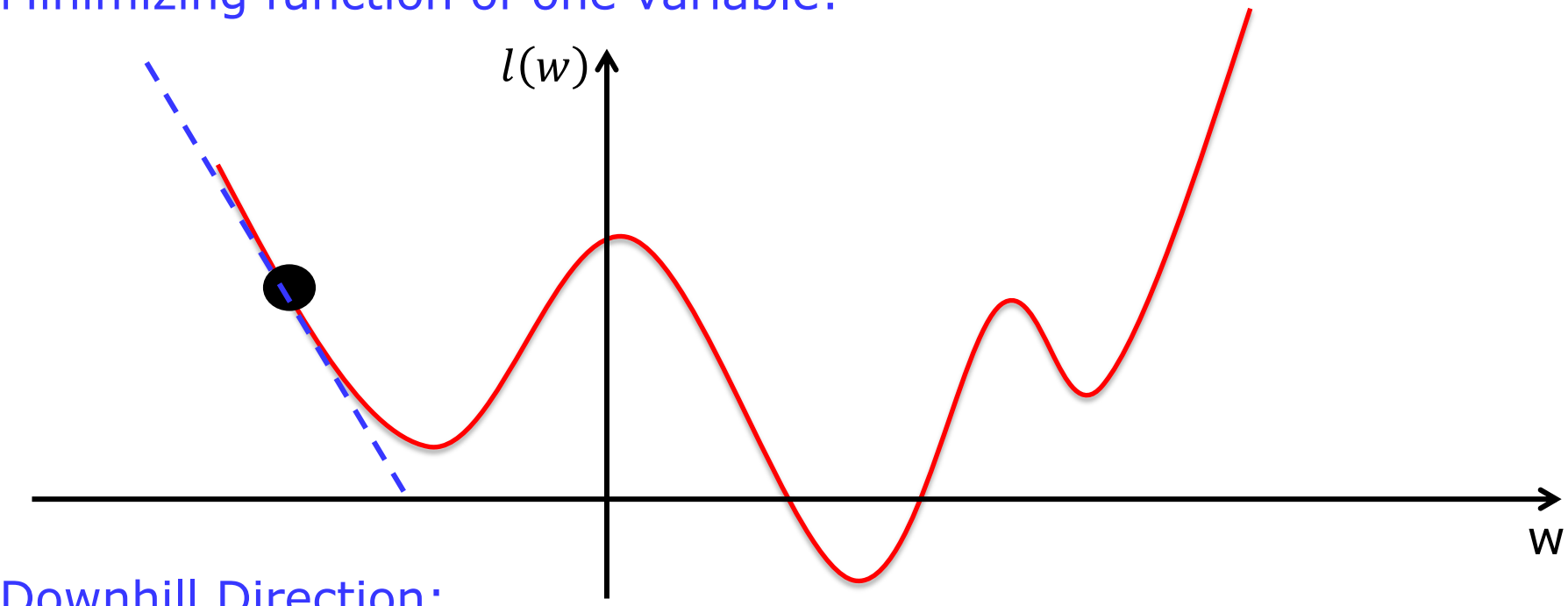
- Minimizing function of one variable:



- Downhill Direction:
  - Compute gradient $\nabla l(w)$ at w.
  - Step in opposite direction of gradient. Step size η.
    - w = w - $\eta \nabla l(w)$

Gradient Descent

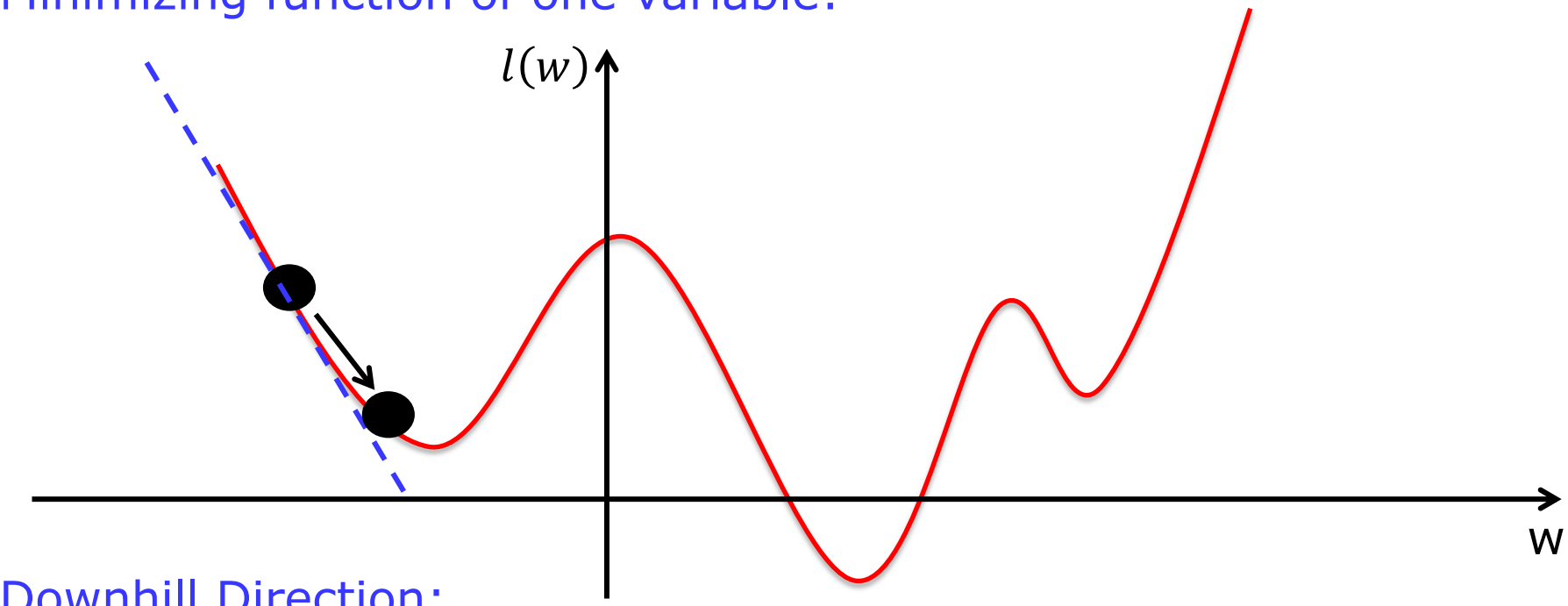
- Minimizing function of one variable:



- Downhill Direction:
  - Compute gradient $\nabla l(w)$ at w.
  - Step in opposite direction of gradient. Step size η.
    - w = w - $\eta \nabla l(w)$

Gradient Descent

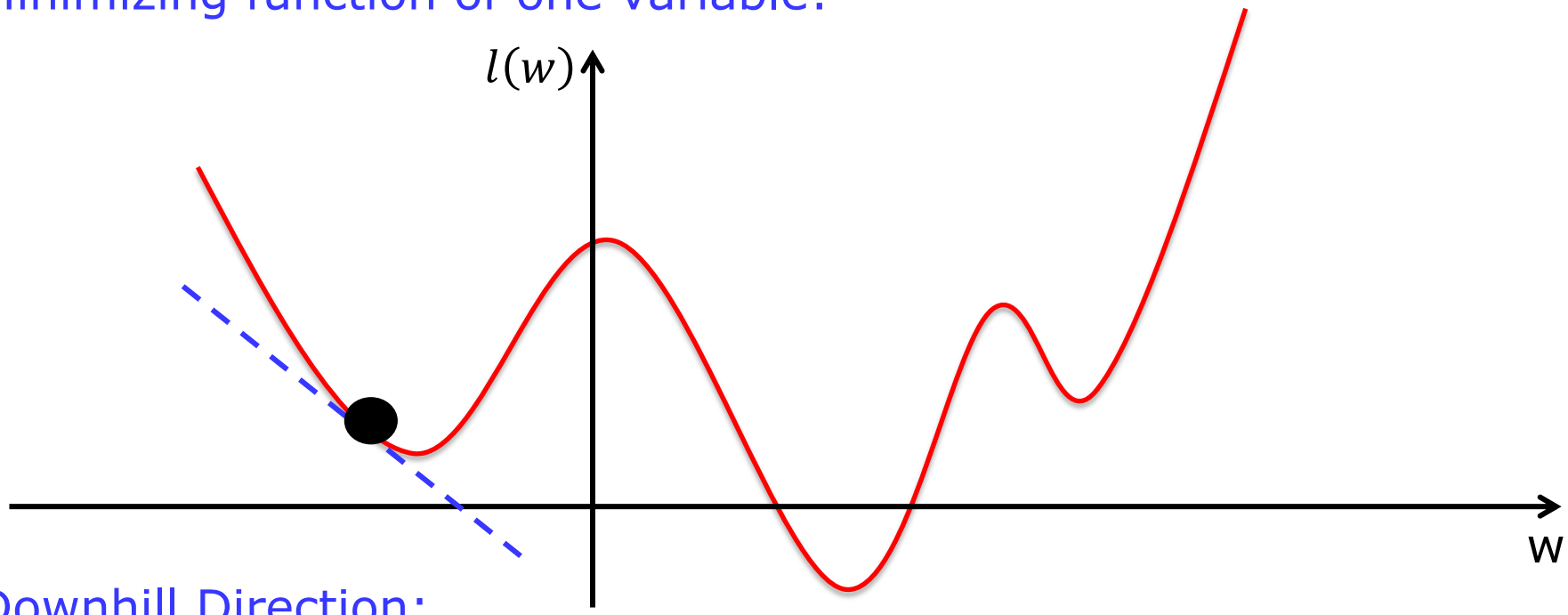
- Minimizing function of one variable:



- Downhill Direction:
  - Compute gradient $\nabla l(w)$ at w.
  - Step in opposite direction of gradient. Step size η.
    - $w = w - \eta \nabla l(w)$

Gradient Descent

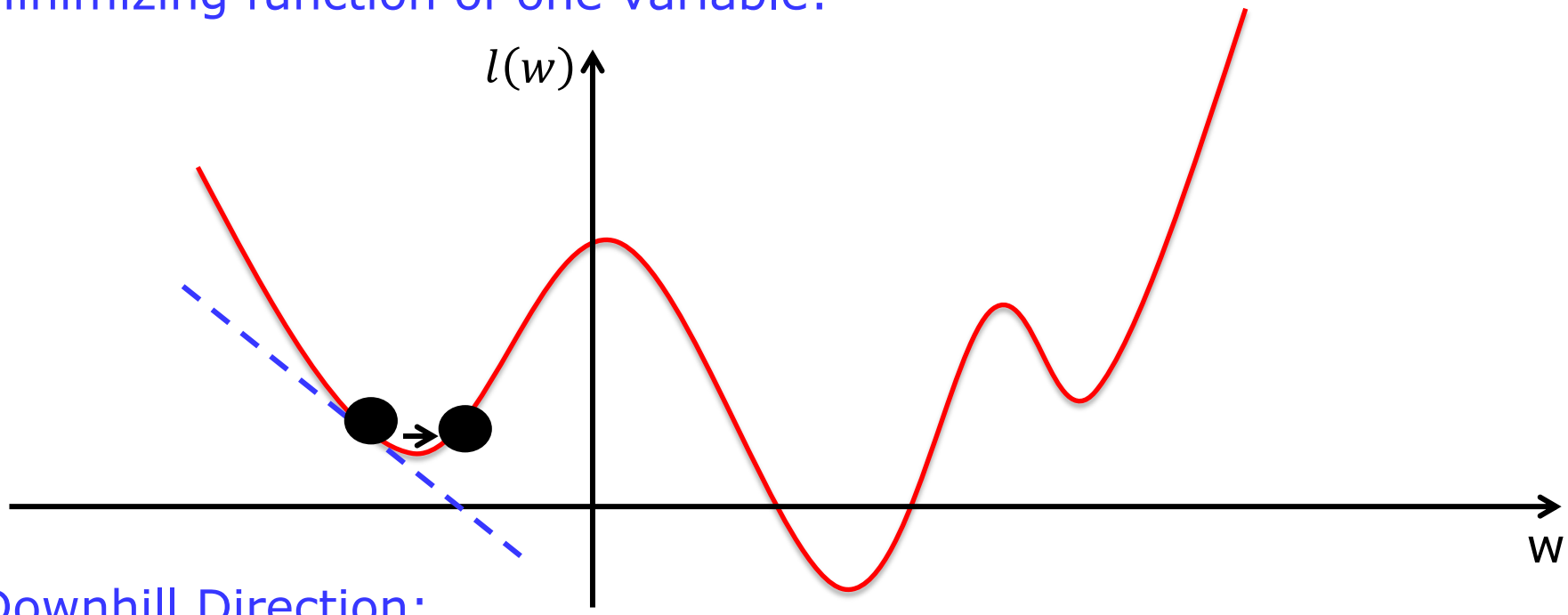
- Minimizing function of one variable:



- Downhill Direction:
  - Compute gradient $\nabla l(w)$ at w.
  - Step in opposite direction of gradient. Step size η.
    - w = w - $\eta \nabla l(w)$

Gradient Descent

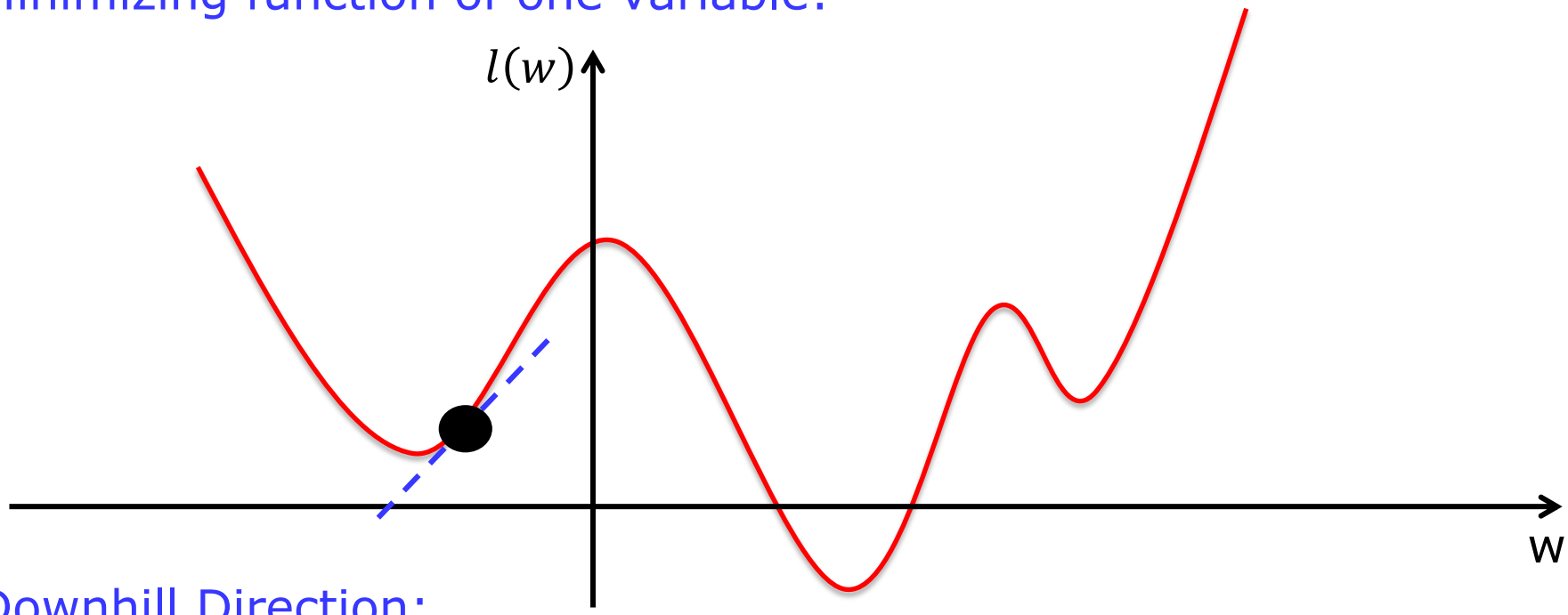
- Minimizing function of one variable:



- Downhill Direction:
  - Compute gradient $\nabla l(w)$ at w.
  - Step in opposite direction of gradient. Step size η.
    - w = w - $\eta \nabla l(w)$

Gradient Descent

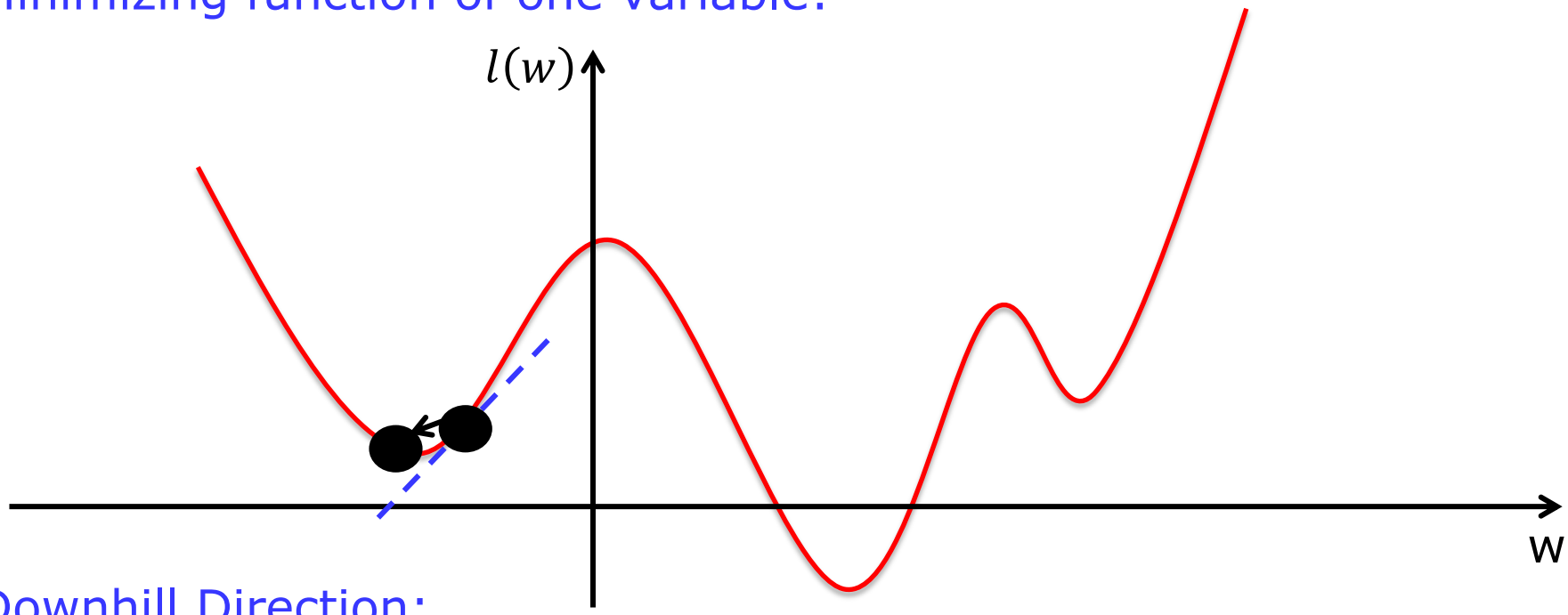
- Minimizing function of one variable:



- Downhill Direction:
  - Compute gradient $\nabla l(w)$ at w.
  - Step in opposite direction of gradient. Step size η.
    - w = w - $\eta \nabla l(w)$

Gradient Descent

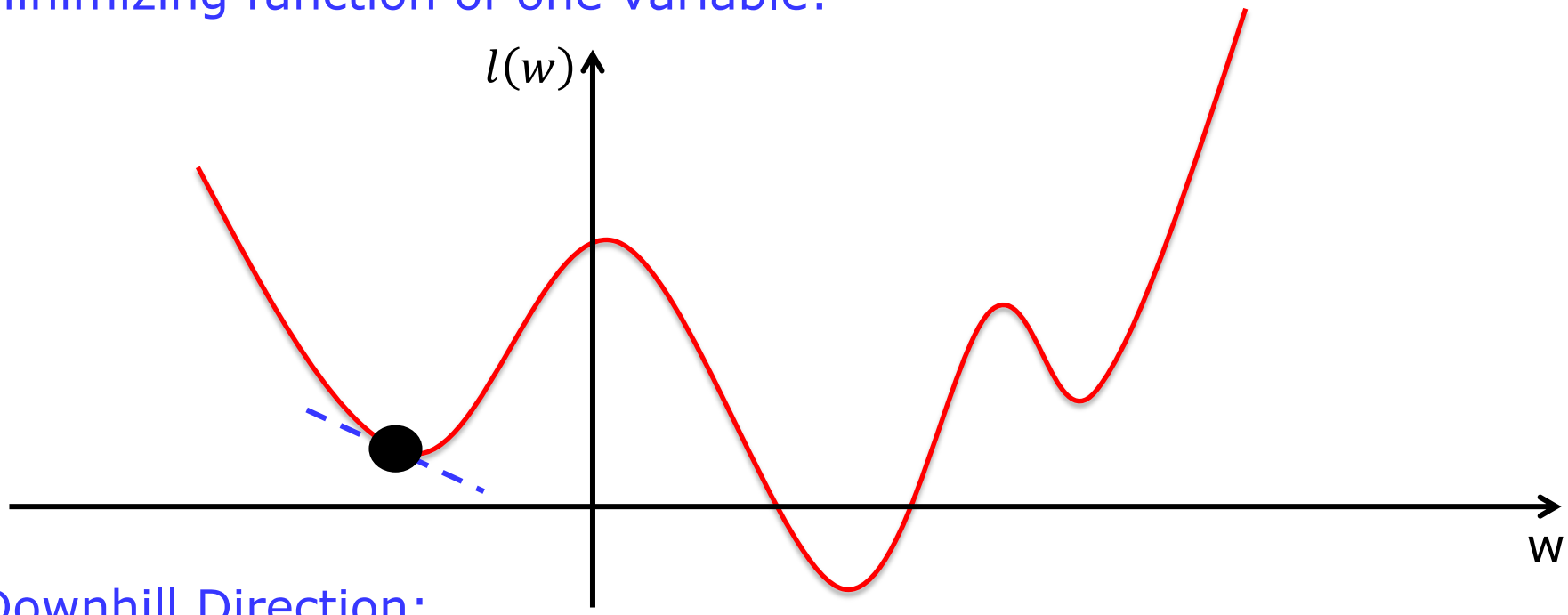
- Minimizing function of one variable:



- Downhill Direction:
  - Compute gradient $\nabla l(w)$ at w.
  - Step in opposite direction of gradient. Step size η.
    - w = w - $\eta \nabla l(w)$

Gradient Descent

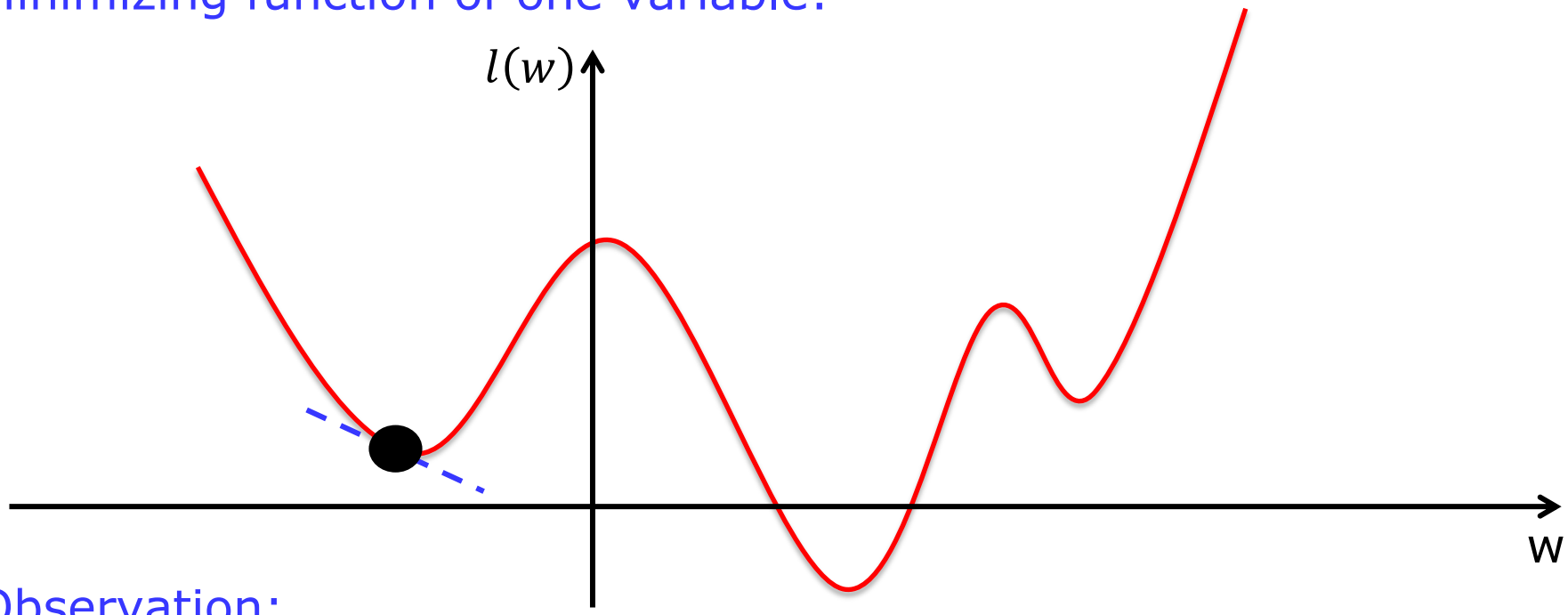
- Minimizing function of one variable:



- Downhill Direction:
  - Compute gradient $\nabla l(w)$ at w.
  - Step in opposite direction of gradient. Step size η.
    - w = w - $\eta \nabla l(w)$

Gradient Descent

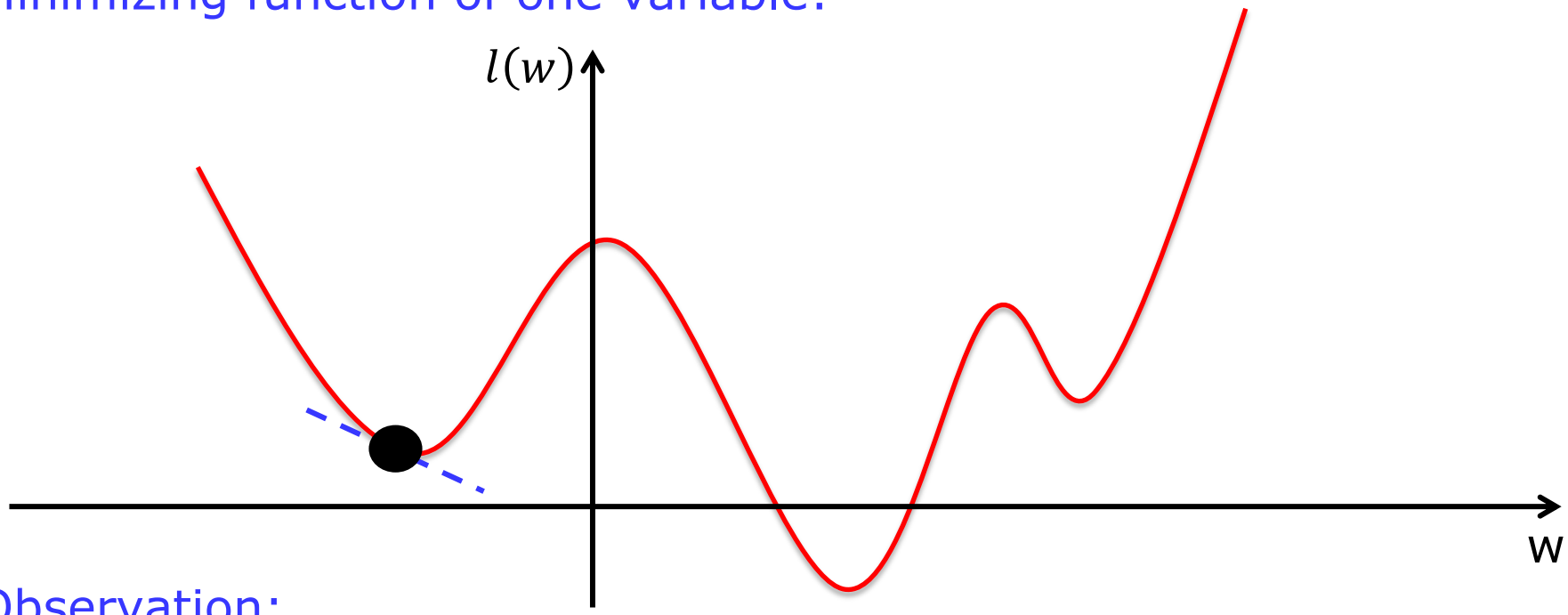
- Minimizing function of one variable:



- Observation:
  - At bottom $\nabla l(w)$ gets closer and closer to 0.
    - Steps get smaller and smaller.

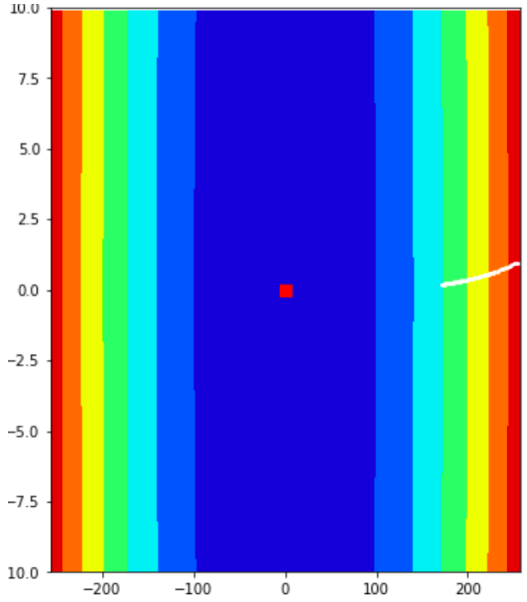
Gradient Descent

- Minimizing function of one variable:

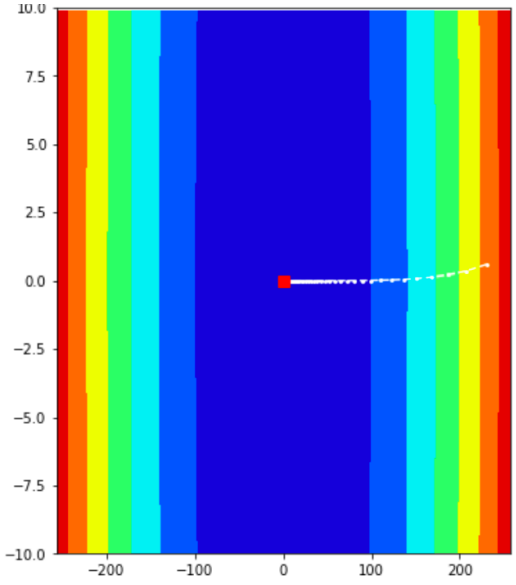


- Observation:
  - If step size η too large, overshoot the minimum.
  - If step size η too small, takes a long time to get there.

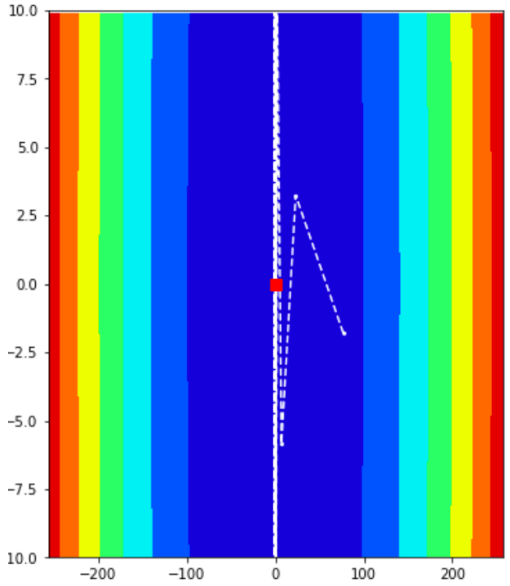
Step Size



Too small η



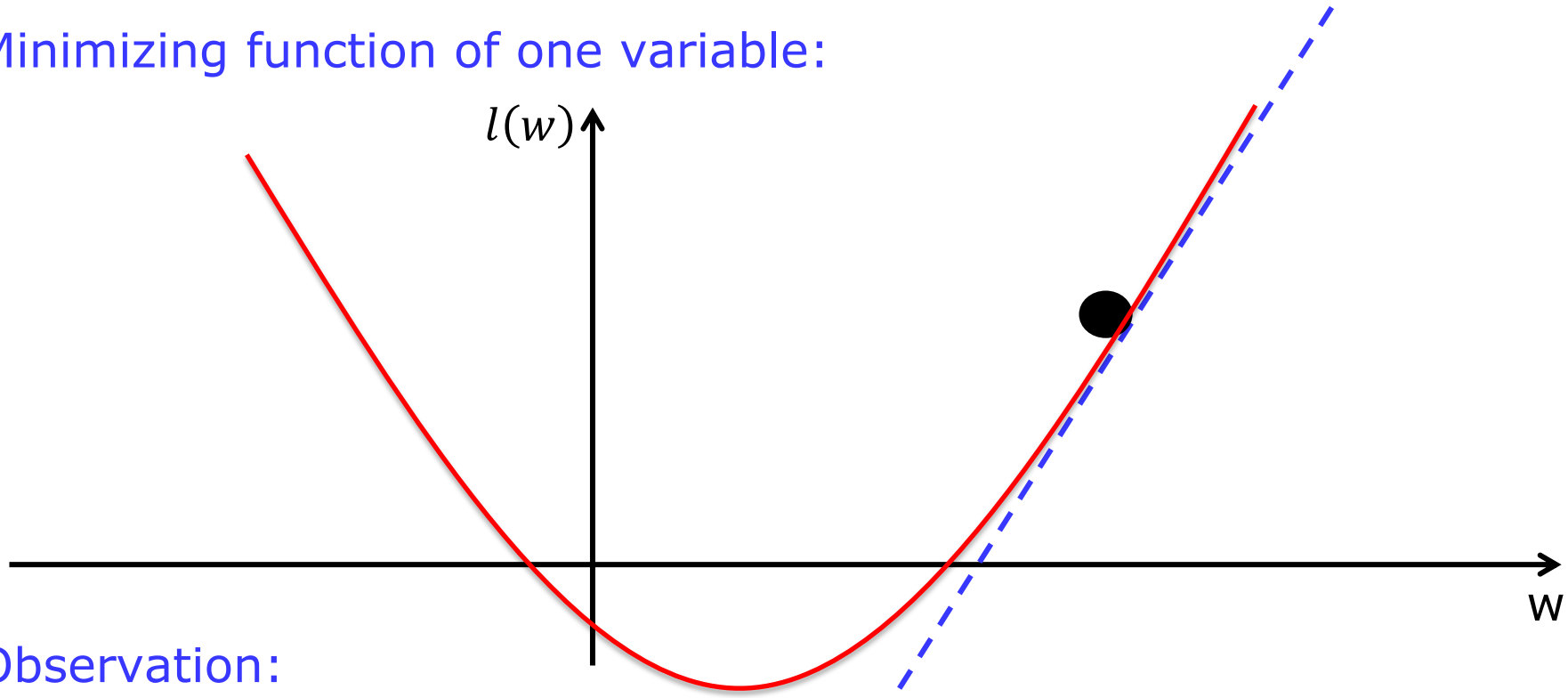
About right



Too large η

Gradient Descent

- Minimizing function of one variable:



- Observation:
  - If convex, only one minimum!
  - Always move towards minimum!

Gradient Descent

- Minimizing function of multiple variables:
- $-\nabla l(w)$ is direction of steepest descent.
  - Walk downhill. Step size η.
    - w = w - $\eta \nabla l(w)$.
- Step size η also called the learning rate.

Practical Observation:
η =0.1 often good.

- Fixed Learning Rate Gradient Descent:
  - Initialize w.
  - While more time:
    - w = w - $\eta \nabla l(w)$
  - Return w.

Variations of gradient descent where η not fixed.

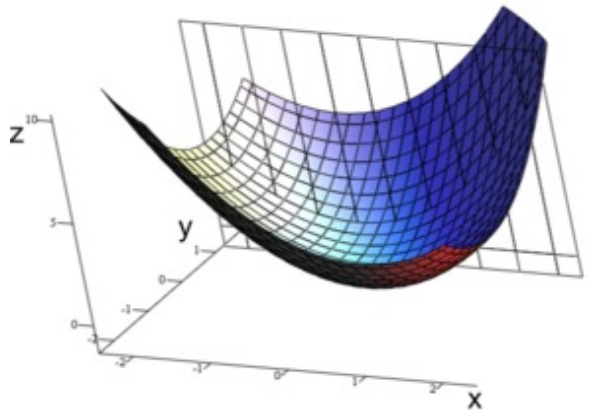
Gradient Descent

- Gradient Descent for Logistic Regression:
  - Initialize w (e.g. random N(0,1) distributed entries).
  - While more time:
    - $\nabla \mathrm{E}_{\mathrm{in}}(w) = \frac{1}{n}\sum_{i=1}^{n} -y_i x_i \, \theta(-y_i w^T x_i)$
    - w = w - $\eta \nabla E_{in}(w)$
  - Return w.

Running time:

O(nd) time to compute.

- Each step takes linear time.



- Observation:
  - Logistic regression loss is convex!
  - There is only one minimum!
  - We move towards it!

Stochastic Gradient Descent

- Stochastic Gradient Descent (SGD) for Logistic Regression:
  - Initialize w (e.g. random N(0,1) distributed entries).
  - While more time:
    - Sample (x,y) uniformly from $(x_1,y_1),\ldots,(x_n,y_n)$.
    - $g = -yx\theta(-yw^T x)$
    - $w = w - \eta g$
  - Return w.

- Each step now takes O(d) time!
- Downside:
  - Only decrease one term in $E_{in}(w) = \frac{1}{n}\sum_{i=1}^{n} \ln(1 + e^{-y_i w^T x_i})$
  - Might make other terms bigger.

Stochastic Gradient Descent

- Stochastic Gradient Descent (SGD) for Logistic Regression:
  - Initialize w (e.g. random N(0,1) distributed entries).
  - While more time:
    - Sample (x,y) uniformly from $(x_1,y_1),\ldots,(x_n,y_n)$.
    - $g = -yx\theta(-yw^T x)$
    - $w = w - \eta g$
  - Return w.

- Observation:
  - Correct gradient in expectation:
    - $E_{x,y}[-yx\theta(-yw^T x)] = \frac{1}{n}\sum_{i=1}^{n} -y_i x_i\, \theta(-y_i w^T x_i) =$
    - $\nabla E_{in}(w)$

Stochastic Gradient Descent

- Stochastic Gradient Descent (SGD) for Logistic Regression:
  - Initialize w (e.g. random N(0,1) distributed entries).
  - While more time:
    - Sample (x,y) uniformly from $(x_1,y_1),\ldots,(x_n,y_n)$.
    - $g = -yx\theta(-yw^T x)$
    - $w = w - \eta g$
  - Return w.

- In Practice:
  - Something in-between SGD and Gradient Descent.
  - Mini-batch Stochastic Gradient Descent.

Stochastic Gradient Descent

- Mini-batch SGD for Logistic Regression:
  - Initialize w (e.g. random N(0,1) distributed entries).
  - While more time:
    - Sample $(x_1', y_1'), \ldots, (x_B', y_B')$ uniformly from $(x_1, y_1), \ldots, (x_n, y_n)$.
    - $g = \frac{1}{B} \sum_{i=1}^{B} -y_i' x_i' \theta(-y_i' w^T x_i')$
    - $w = w - \eta g$
  - Return w.

- Observation:
  - Like SGD, but compute gradient over B points.
  - Reduces variance.
  - B often between 4 and 64 (rule of thumb).

Stochastic Gradient Descent

- Mini-batch SGD for Logistic Regression:
  - Initialize w (e.g. random N(0,1) distributed entries).
  - While more time:
    - Sample $(x_1', y_1'), \ldots, (x_B', y_B')$ uniformly from $(x_1, y_1), \ldots, (x_n, y_n)$.
    - $g = \frac{1}{B} \sum_{i=1}^{B} -y_i' x_i' \theta(-y_i' w^T x_i')$
    - $w = w - \eta g$
  - Return w.

- In Practice on Large Data Sets:
  - Random access to B elements too expensive.
  - Instead shuffle n samples randomly and run through in batches of B elements.

Stochastic Gradient Descent

- Mini-batch SGD for Logistic Regression:
  - Initialize w (e.g. random N(0,1) distributed entries).
  - While more time:
    - Permute $(x_1,y_1),\ldots,(x_n,y_n)$ randomly.
    - For i=1,...,n/B:
      - Let $(x_1',y_1'),\ldots,(x_B',y_B')$ be next B data items.
      - $g = \frac{1}{B}\sum_{i=1}^{B} -y_i' x_i' \theta(-y_i' w^T x_i')$
      - w = w - ηg
  - Return w.

- Each time we permute is called an epoch.
- Practical: Reduce η by a small factor after fixed number of epochs.

Stochastic Gradient Descent

- Any point-wise loss:
  - $E_{in}(w) = \frac{1}{n}\sum_{i=1}^{n} L(w(x_i), y_i)$
- Mini-batch SGD:
  - Initialize w (e.g. random N(0,1) distributed entries).
  - While more time:
    - Permute $(x_1,y_1),\ldots,(x_n,y_n)$ randomly.
    - For i=1,...,n/B:
      - Let $(x_1',y_1'),\ldots,(x_B',y_B')$ be next B data items.
      - $g = \frac{1}{B}\sum_{i=1}^{B} \nabla L(w(x_i'), y_i')$
      - $w = w - \eta g$
  - Return w.

Mini-Batch SGD

It is Used Everywhere!







Tensorflow and all other deep learning systems

Integral in Sklearn package



Summary Logistic Regression

- Likelihood of training data.
  - Hypothesis w, i.e. $\theta(w^T x) = \frac{1}{1+e^{-w^T x}}$ should equal Pr[y=1|x].
- Likelihood of data labels with $S=(x_1,y_1),\ldots,(x_n,y_n)$ for w:
  - $\Pr[S \mid w,X] = \prod_{i=1}^{n} \frac{1}{1+e^{-y_i w^T x_i}}$

- Maximum Likelihood:
  - Maximize Pr[S | w,X] same as minimize –ln(Pr[S | w,X]).
  - Showed –ln(Pr[S | w,X]) is convex.
  - Computed gradient of –ln(Pr[S | w,X]).
  - Use Gradient Descent to minimize.

Logistic Regression

